Einführung in die Rechnerarchitektur (ERA) IN0004

Martin Schulz
schulzm@in.tum.de
Chair for Computer Architecture and Parallel Systems
https://www.ce.cit.tum.de/caps/

Robert Wille
robert.wille@tum.de
Chair for Design Automation
https://www.cda.cit.tum.de/



Willkommen im Bachelor Studiengang Informatik

Informatik (B.Sc.) – von www.in.tum.de

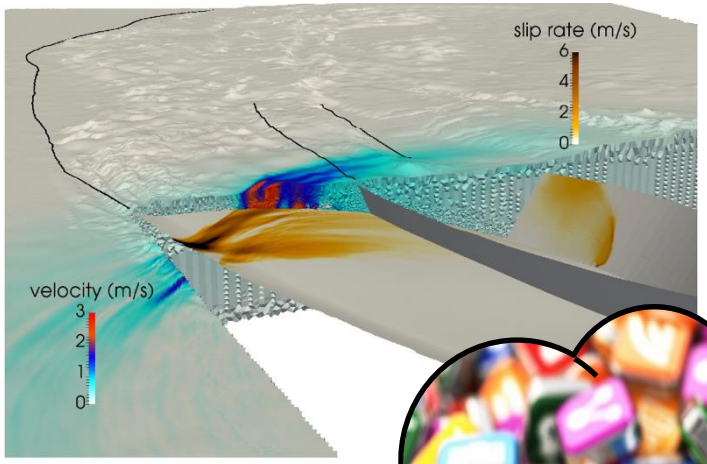
> Informatik ist die Wissenschaft der systematischen, automatisierten Verarbeitung von Informationen. Sie ist eine sich ständig weiterentwickelnde Disziplin, die sich damit beschäftigt, wie Informationen in ursprünglichen und auch hochtechnisierten Computersystemen strukturiert, präsentiert und verarbeitet werden.

Vorlesungen im ersten Semester

- Einführung in die Informatik 1
- Algorithmen, Programmkonstrukte, Datenstrukturen, Rekursion
- Diskrete Strukturen
- Mengen, Relationen, Aussagelogik, Kombinatorik, Graphentheorie
- Praktikum Grundlagen der Programmierung

- **Einführung in die Rechnerarchitektur**

Von Code zu Ergebnissen



Erdbeben Simulation mit SeisSol (TUM&LMU)



```
int pos;
char *cmdargv[MAX_CMDARGS + 1];
int cmdargc, comment, i;

/* setup vars */

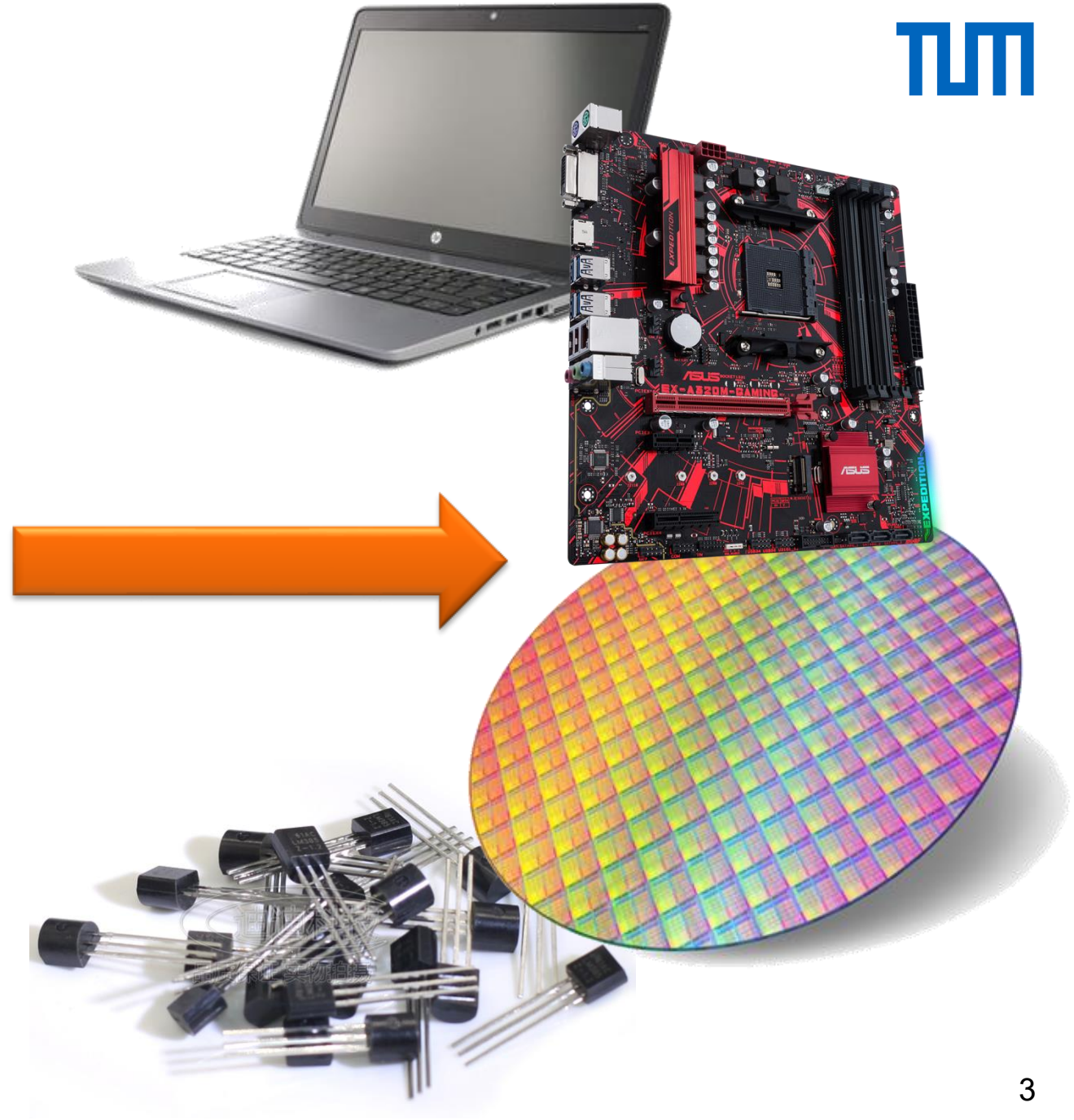
pnmpi_max_level = 0;
pnmpi_initialization_complete = 0;

/* set global defaults */
/* none at this moment */

modules.module = NULL;
modules.num = 0;
modules.numalloc = 0;
modules.pcontrol = PNMPI_PCONTROL_INT;

/* locate library */

lib_path_string = getenv("PNMPI_LIB_PATH");
if (lib_path_string == NULL)
  {
    // no user lib_path_string; just use the install destination's module
    // path.
    lib_path_string = PNMPI_MODULES_DIR;
  }
else
  {
    // concat the user lib_path_string with the install destination's module
    // path.
    size_t len = strlen(lib_path_string) + strlen(PNMPI_MODULES_DIR) + 2;
    const char *old_lib_path_string = lib_path_string;
    lib_path_string = (char *)alloca(len * sizeof(char));
    sprintf(lib_path_string, "%s:%s", old_lib_path_string, PNMPI_MODULES_DIR);
  }
library_path = parse_path(lib_path_string);
DBGPRINT2("Library path is: %s", lib_path_string);
```



Kernthemen – Wie führt ein Rechner Programmer aus?

Maschinennahe Repräsentierung von Daten und Programmen
- Kodierungen
- Binär-Repräsentationen
- Assemblersprachen

Struktur von Rechensystemen
- Komponenten die Daten verarbeiten
- Komponenten die Daten speichern
- Komponenten die Daten kommunizieren

Umsetzung von Komponenten
- Steuerung einzelner Subkomponenten → Einzelne Steuer und Datensignale
- Beschreibung einzelner Komponenten → Spezielle Programmiersprachen
- Implementierung durch Schaltwerke und Schaltnetze → Direkte Übersetzung in Transistoren

Moderne Rechnerarchitekturen
- Speichersysteme
- Parallele Systeme

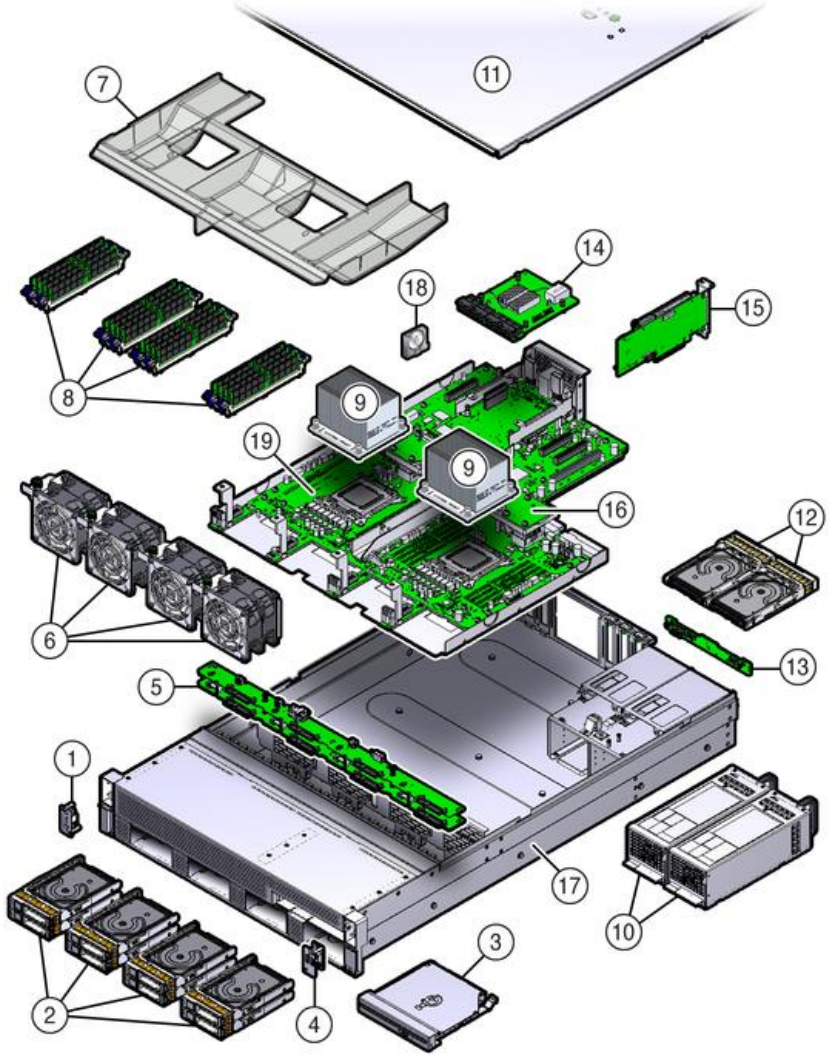
Warum brauche ich das?

Rechnerarchitektur ist ein breites Thema an sich
- Entwicklung von modernen Prozessoren
- Entwicklung von eingebetteten Spezialsystemen
- Systemdesign

Zunehmende Spezialisierung von Rechnern

Grundverständnis über Rechnerstrukturen
- Auswahl von passenden Plattformen
- Design von effizienten Algorithmen
- Energieeffizienz

Optimierungen
- Abbildung von Algorithmen auf Architekturen
- Verständnis von möglichen Engpässen
- „Performance Portability“



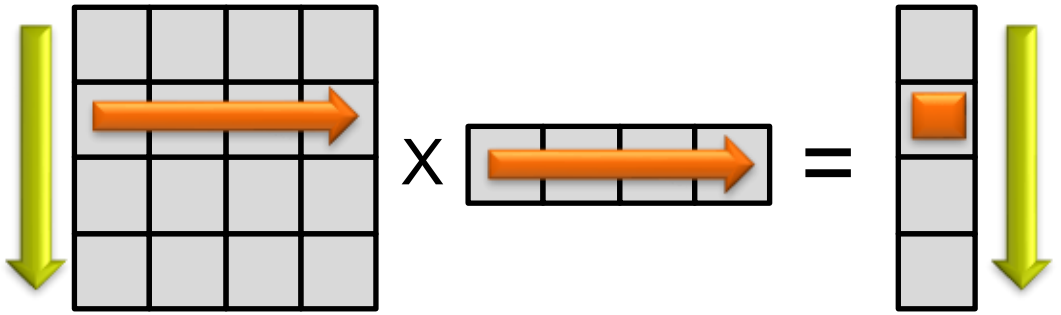
From: https://docs.oracle.com/cd/E36974_01/html/E38145/z400000a1419520.html

Beispiel Matrix/Vektormultiplikation

Matrix A, Vektor v und b

$$v = A*b$$

$$v_i = \sum_j A_{ij} * b_j$$



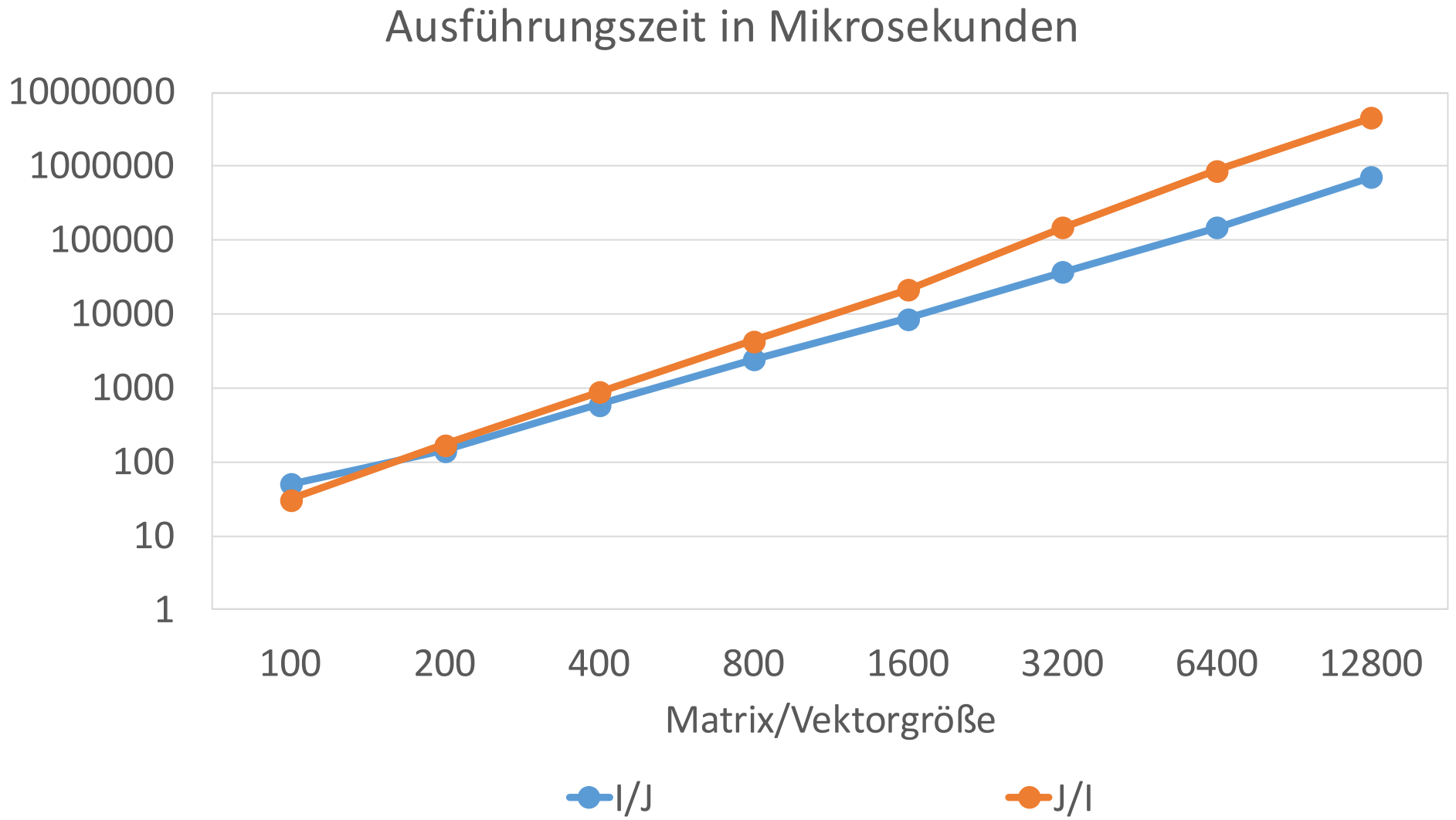
```
for (i=0; i<N; i++)
  for (j=0; j<N; j++)
    v[i]=v[i]+A[i][j]*b[j];
```

N=12.800
Zeit: 0,71s

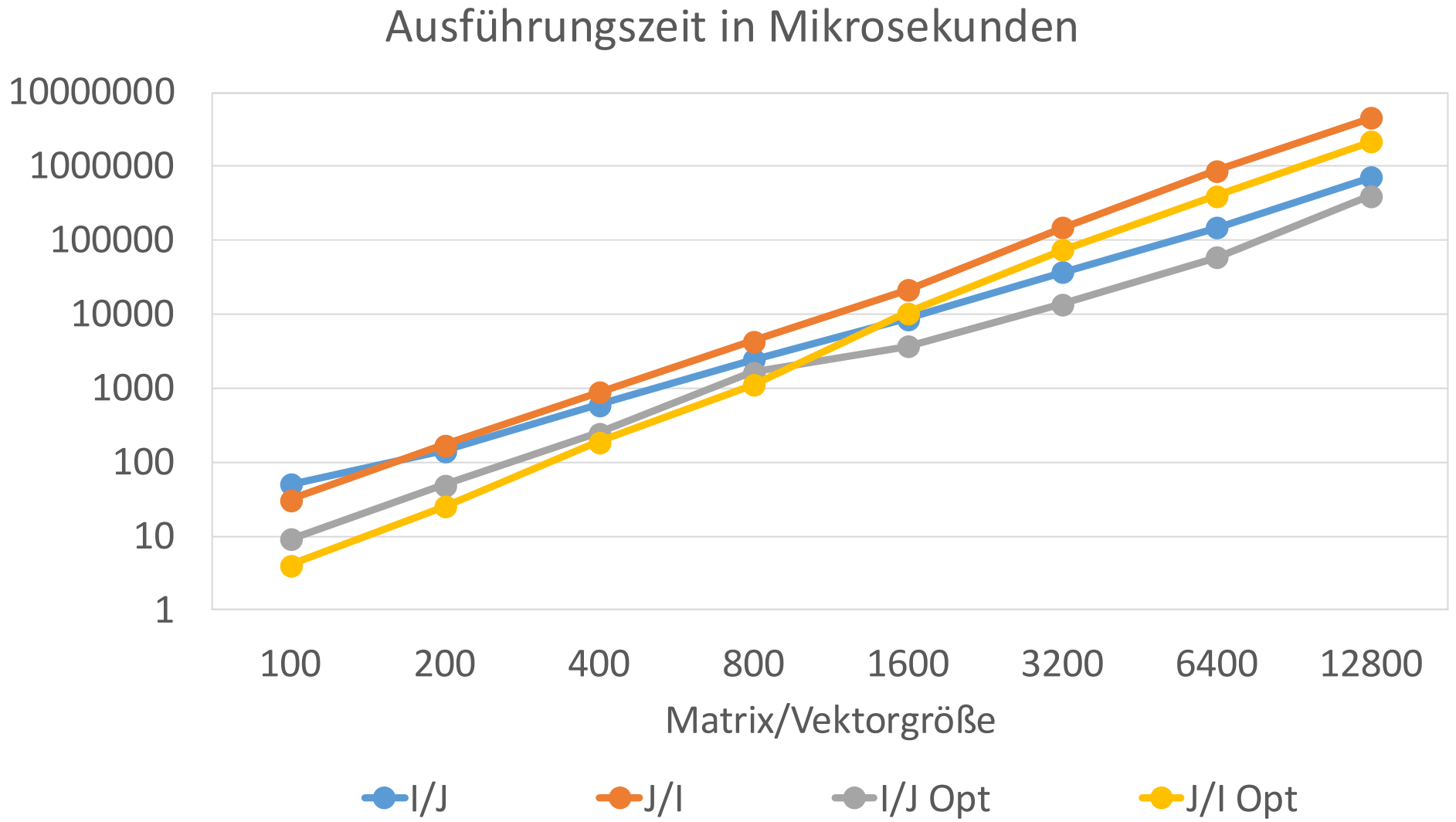
```
for (j=0; j<N; j++)
  for (i=0; i<N; i++)
    v[i]=v[i]+A[i][j]*b[j];
```

N=12.800
Zeit: 4,54s

Beispiel Matrix/Vektormultiplikation



Beispiel Matrix/Vektormultiplikation



Inhalts Übersicht

Block 1: Assembler Programmierung
- Repräsentationen
- Instruktionsarten und Umsetzungen

Block 2: Speicher
- Virtueller Speicher
- Implementierung
- Caches

Block 3: Parallelismus
- In-Core parallelismus
- Multicore Systeme

Block 4: E/A und Peripherie Komponenten

Block 5: Implementierung von Speichern und ICs

Block 6: Prozessor Implementierung
- Boolesche Algebra
- Einfache Schaltungen
- Basics eines Prozessors

Block 7: Computer Design
- Logikoptimierung
- Synthese

Block 8: Andere Architekturen
- Quantum Computing

Reihenfolge und genaue Inhalte können sich ändern!

Einführung in die Rechnerarchitektur (ERA)
IN0004
Logistik

Martin Schulz
schulzm@in.tum.de
Chair for Computer Architecture and Parallel Systems
https://www.ce.cit.tum.de/caps/

Robert Wille
robert.wille@tum.de
Chair for Design Automation
https://www.cda.cit.tum.de/



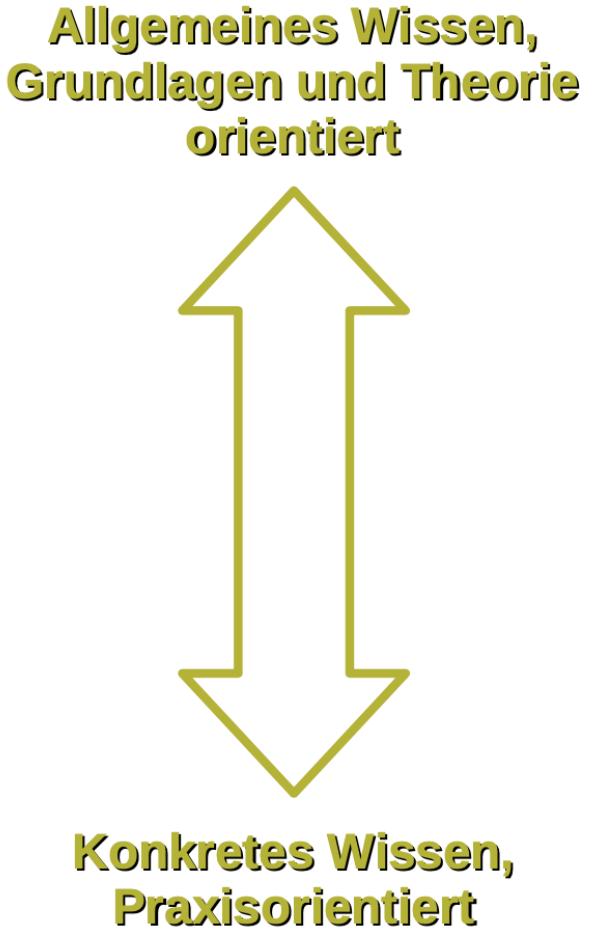
Dreistufiges Lernkonzept

Lernkonzept in drei Schichten

Vorlesung
- Übersicht von Rechnerarchitekturen
- Basiswissen und Hintergrund
- Allgemeines Verständnis

Zentralübung
- Vertiefung und Wiederholung
- Praxisnähere Umsetzung der Vorlesung

Tutorübungen
- Kleine Gruppen für Nachfragen
- Besprechung von Übungsblättern
- “Hands-on”



ERA Team

Vorlesung

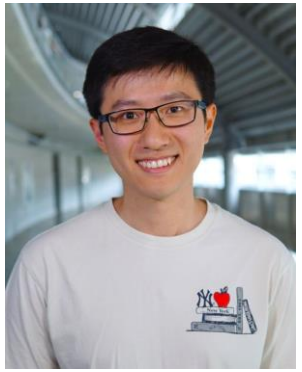
- Prof. Dr. Martin Schulz
- Prof. Dr. Robert Wille




Zentralübungen und Übungsbetrieb

- Benjamin Hien
- Dominic Prinz
- Jakob Schäffeler
- Julian Shen



Großes Tutorenteam:
Jana Brade, Leopold Cario, Julian Dunkel, Markus Gschoßmann, Mariam Kharrat, Niklas Ladurner, Nico Marikucza, Simon Preibisch, Arvin Vach, Tobias Vale, Jonathan Woltereck

Logistik Vorlesung

Zeiten: Montag, 12:00 (s.t.) – 13:30
Dienstag, 16:00 (s.t.) – 17:30

Ort: Audimax im Galileo
(im Notfall durch Live Stream ersetzt)

Wir werden versuchen alle Vorlesungen aufzuzeichnen

- Live Stream und Aufzeichnung http://live.rbg.tum.de/ (Link in Moodle)

→ Aber: „best effort only“

Skript/Folien (jede Woche)
http://moodle.tum.de/ -> IN-0004
Login mit TUM-Online Kennung

Änderungen (falls nötig) werden auf Moodle angekündigt.

Logistik Zentral-Übung

Zeit: Freitag, 13:15 – 14:45

Ort: Ernst-Schmidt-Hörsaal im MW 1801

Wir werden versuchen alle Zentralübungen aufzuzeichnen und

- Live Stream und Aufzeichnung http://live.rbg.tum.de/ (Link in Moodle)

→ Aber: „best effort only“

Material über Moodle:

http://moodle.tum.de/ -> IN-0004

Erste Zentralübung: am 16.10. (diesen Freitag)

Logistik Übungsgruppen

Kleine Gruppen geleitet von studentischen Tutoren
(Inhalte unterschiedlich von Zentralübung)

Material über Moodle:

http://moodle.tum.de/ -> IN-0004
(Lösungsvorschläge zum Übungsblatt eine Woche später)

Anmeldung in TUM-Online nötige: **BITTE SELBST ANMELDEN**

Nähere Angaben: Freitag in der ZÜ

Beginn der Tutorübungen: Montag, 19.10.

Wöchentliche Hausaufgaben via Artemis

ERA Klausur

Schriftliche Prüfung

- 120min
- Anmeldung über TUM-Online erforderlich !!!

ERA Klausur

Schriftliche Prüfung

- 120min
- Anmeldung über TUM-Online erforder

Klausurrelevanz



ERA Klausur

Schriftliche Prüfung

- 120min
- Anmeldung über TUM-Online erforderlich !!!

Klausurrelevanz

- Gesamter (deutschsprachiger) Stoff der LV, ZÜ, TÜ
- Bitte nicht (nur) an alten Klausuren orientieren



ERA Klausur

Schriftliche Prüfung
- 120min
- Anmeldung über TUM-Online erforderlich !!!

Klausurrelevanz
- Gesamter (deutschsprachiger) Stoff der LV, ZÜ, TÜ
- Bitte nicht (nur) an alten Klausuren orientieren

”Closed Book”
- Keine Hilfsmittel (außer Wörterbücher), keine Taschenrechner
- Referenzmaterialen werden zur Verfügung gestellt

Sprache: Deutsch, (konsistente) Antworten in Englisch können akzeptiert werden

Anmeldungsfrist voraussichtlich Mitte November bis Mitte Januar
Termin voraussichtlich: tbd.
(ohne Gewähr und kann sich ändern)
BITTE SELBST NACHSCHAUEN UND ANMELDEN !!!

Ausblick ERA Praktikum (SoSe)

Voraussetzung

- Erfolgreiches Bestehen der ERA-Klausur
- Verfahren wird zu einem späteren Zeitpunkt noch genauer erläutert

Durchführungszeitraum

- Projekte im SS27
- Anmeldung nach Bekanntgabe der ERA-Klausurnoten

Gruppenaufgaben

- Systemprogrammierung und Optimierungen
- Prozessor-Entwicklung

Kontakt

Alle organisatorischen Fragen an:

- era@lists.lrz.de
- Geht an das ganze ERA Team

Allgemein bei Fragen:

- Tutoren sind die ersten Ansprechpartner (in den Übungen/via Zulip)
  Direkte Zuordnung!
- Fragen und Anmerkungen via Zulip
- Bei offiziellen Anfragen: Übungsleitung (era@lists.lrz.de)

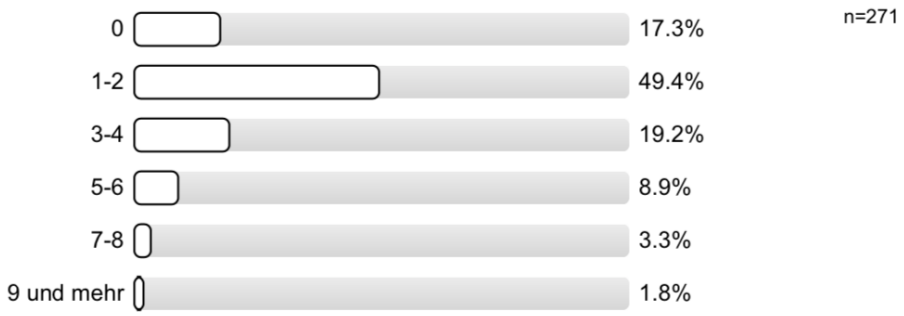
Als letzter Schritt: Dozent

Zulip als Informationskanal (zulip.in.tum.de)

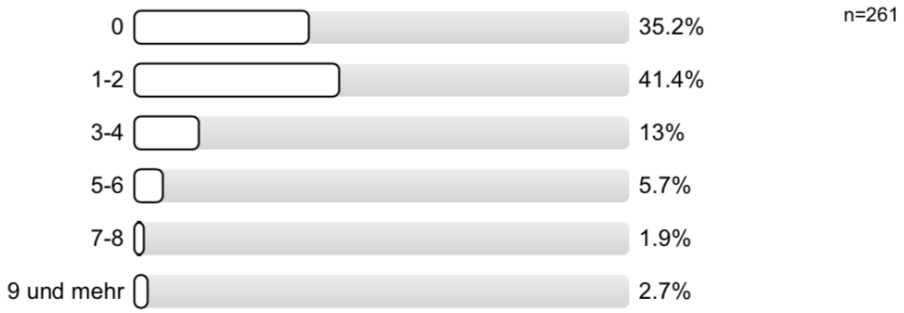
- „ERA - *“ Kanäle sind öffentlich
- Kanäle können via Moodle einfach gefunden werden
- Fragekanal für Vorlesung: „ERA – Vorlesung“
- Mehr Informationen auf Moodle und in der Zentralübung

Anmerkung zur Nachbearbeitungszeit (Daten von der Umfrage WS18/19)

Wie viel Zeit in Stunden pro Woche wenden Sie durchschnittlich für die Vor- und Nachbereitung der Vorlesung (OHNE Zeit für die Hausaufgaben und Prüfungsvorbereitung bzw. Erbringung des Leistungsnachweises) auf?



Wie viel Zeit wenden Sie durchschnittlich für die Hausaufgaben in Stunden pro Woche auf?



ERA = 8 ECTS = 240h Arbeitszeit
(120h Vorlesung, 60h ZÜ, 60h Tutor-Übungen)

Weitere Quellen / Hintergrundmaterial



Rechnerarchitektur:

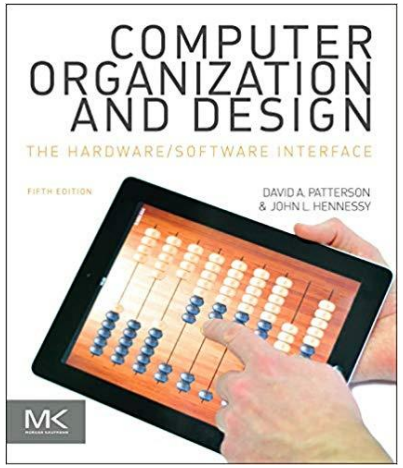
- Von der digitalen Logik zum Parallelrechner
  Tanenbaum & Austin
- Computer Organization and Design
  Patterson & Hennessy
- Digital Design and Computer Architecture, RISC-V Edition
  Harris & Harris

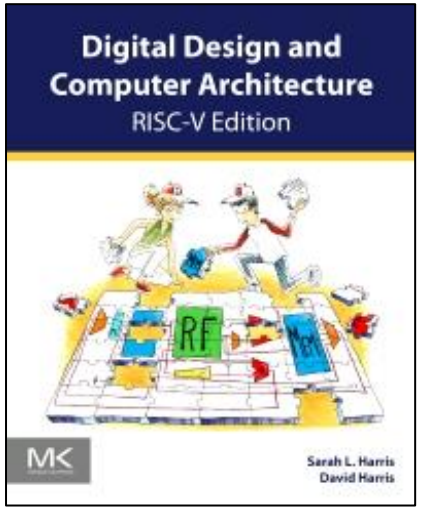
Zusätzliches Material auf Moodle

Links in den Vorlesungsfolien

- Nicht nur als Quellen gedacht
- Oft weiterführende Informationen

Praktische Experimente machen vieles klar





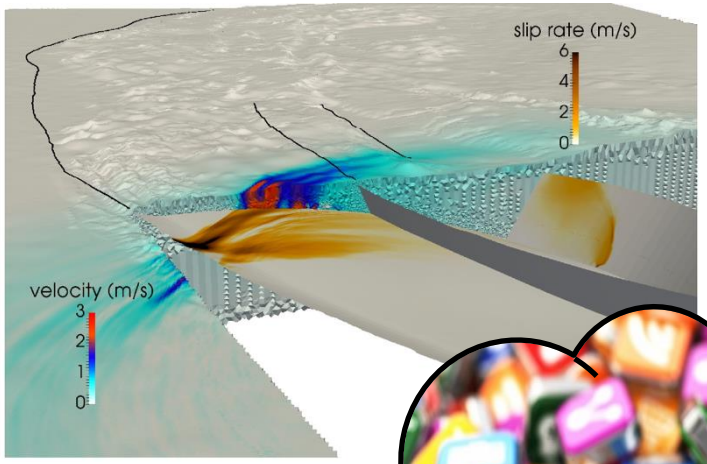
Einführung in die Rechnerarchitektur (ERA)
IN0004
Datendarstellungen

Martin Schulz
schulzm@in.tum.de
Chair for Computer Architecture and Parallel Systems
https://www.ce.cit.tum.de/caps/

Robert Wille
robert.wille@tum.de
Chair for Design Automation
https://www.cda.cit.tum.de/



Von Code zu Ergebnissen



Erdbeben Simulation mit SeisSol (TUM&LMU)



```
int pos;
char *cmdargv[MAX_CMDARGS + 1];
int cmdargc, comment, i;

/* setup vars */

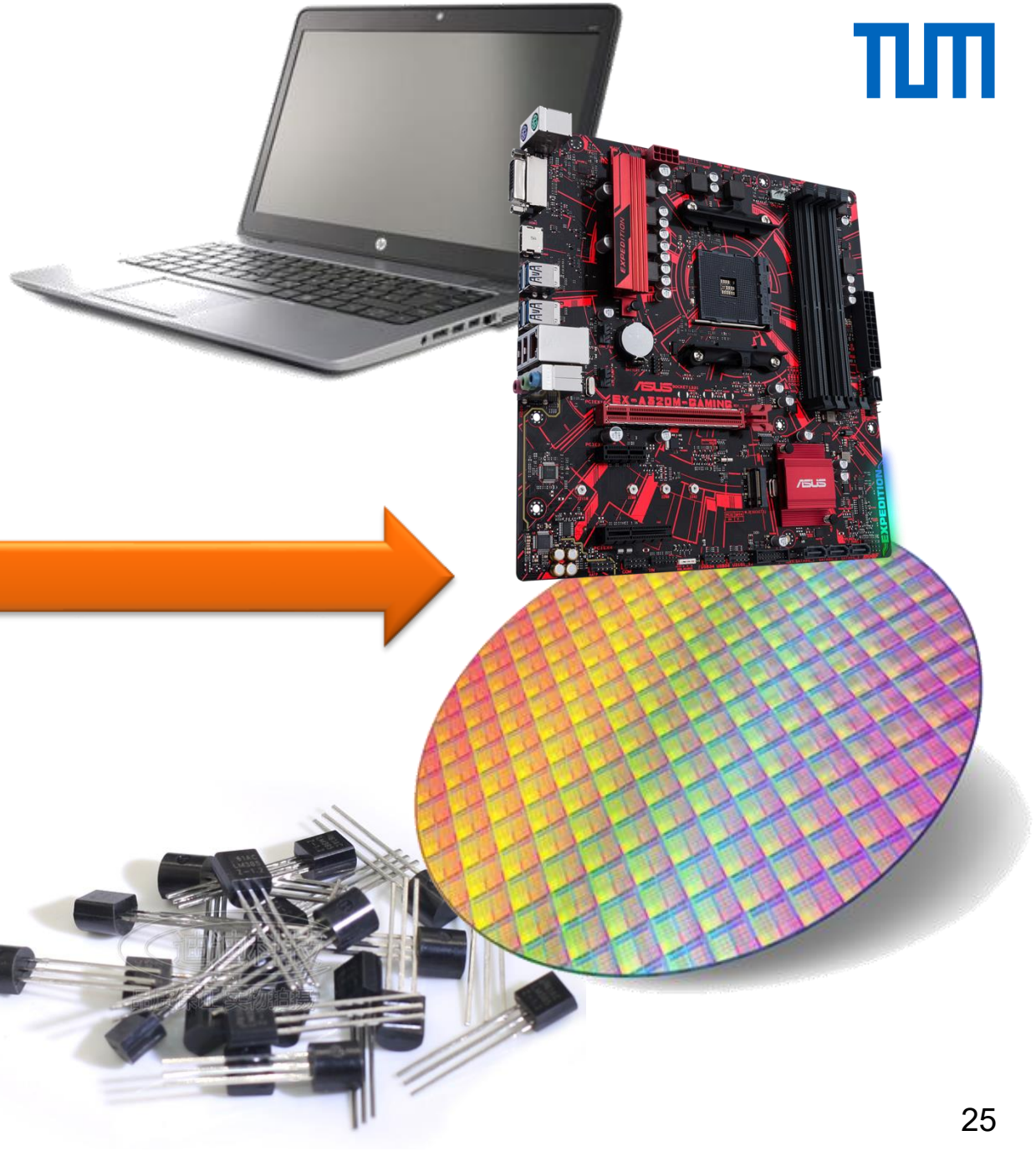
pnmpi_max_level = 0;
pnmpi_initialization_complete = 0;

/* set global defaults */
/* none at this moment */

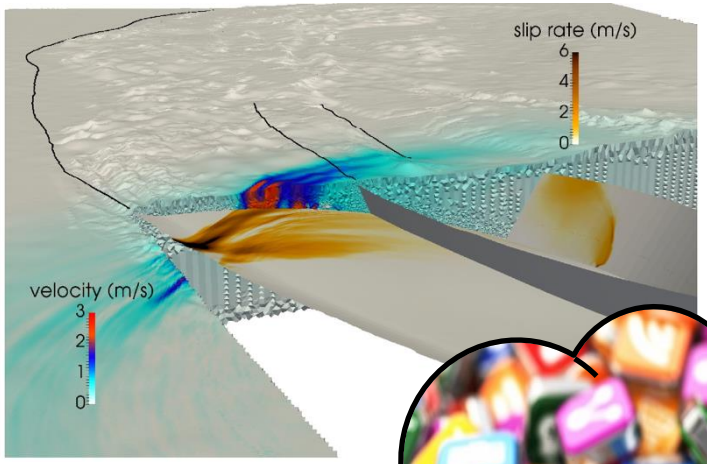
modules.module = NULL;
modules.num = 0;
modules.numalloc = 0;
modules.pcontrol = PNMPI_PCONTROL_INT;

/* locate library */

lib_path_string = getenv("PNMPI_LIB_PATH");
if (lib_path_string == NULL)
  {
    // no user lib_path_string; just use the install destination's module
    // path.
    lib_path_string = PNMPI_MODULES_DIR;
  }
else
  {
    // concat the user lib_path_string with the install destination's module
    // path.
    size_t len = strlen(lib_path_string) + strlen(PNMPI_MODULES_DIR) + 2;
    const char *old_lib_path_string = lib_path_string;
    lib_path_string = (char *)alloca(len * sizeof(char));
    sprintf(lib_path_string, "%s:%s", old_lib_path_string, PNMPI_MODULES_DIR);
  }
library_path = parse_path(lib_path_string);
DBGPRINT2("Library path is: %s", lib_path_string);
```



Von Code zu Ergebnissen



Erdbeben Simulation mit SeisSol (TUM&LMU)



```
int pos;
char *cmdargv[MAX_CMDARGS + 1];
int cmdargc, comment, i;

/* setup vars */

pnmpi_max_level = 0;
pnmpi_initialization_complete = 0;

/* set global defaults */
/* none at this moment */

modules.module = NULL;
modules.num = 0;
modules.numalloc = 0;
modules.pcontrol = PNMPI_PCONTROL_INT;

/* locate library */

lib_path_string = getenv("PNMPI_LIB_PATH");
if (lib_path_string == NULL)
  {
    // no user lib_path_string; just use the install destination's module
    // path.
    lib_path_string = PNMPI_MODULES_DIR;
  }
else
  {
    // concat the user lib_path_string with the install destination's module
    // path.
    size_t len = strlen(lib_path_string) + strlen(PNMPI_MODULES_DIR) + 2;
    const char *old_lib_path_string = lib_path_string;
    lib_path_string = (char *)alloca(len * sizeof(char));
    sprintf(lib_path_string, "%s:%s", old_lib_path_string, PNMPI_MODULES_DIR);
  }
library_path = parse_path(lib_path_string);
DBGPRINT2("Library path is: %s", lib_path_string);
```



Wie kann ich Eingaben repräsentieren?

- Speicherung im System
- Einfache Ausführung von Berechnungen
- Interpretation im System

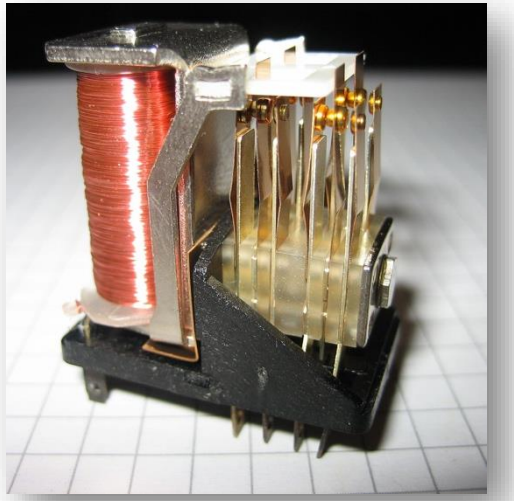
Vielseitige Eingaben

- Ganze Zahlen
- Reale Zahlen
- Texte

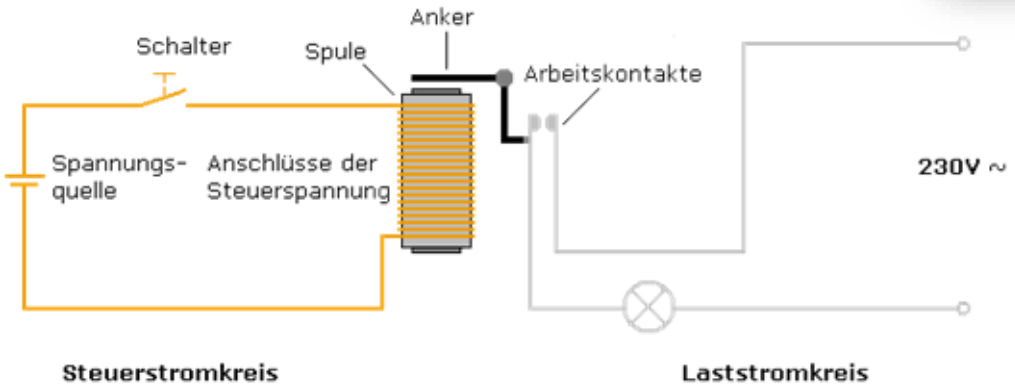
Datendarstellung

Rechner arbeiten (meistens) im Binärsystem

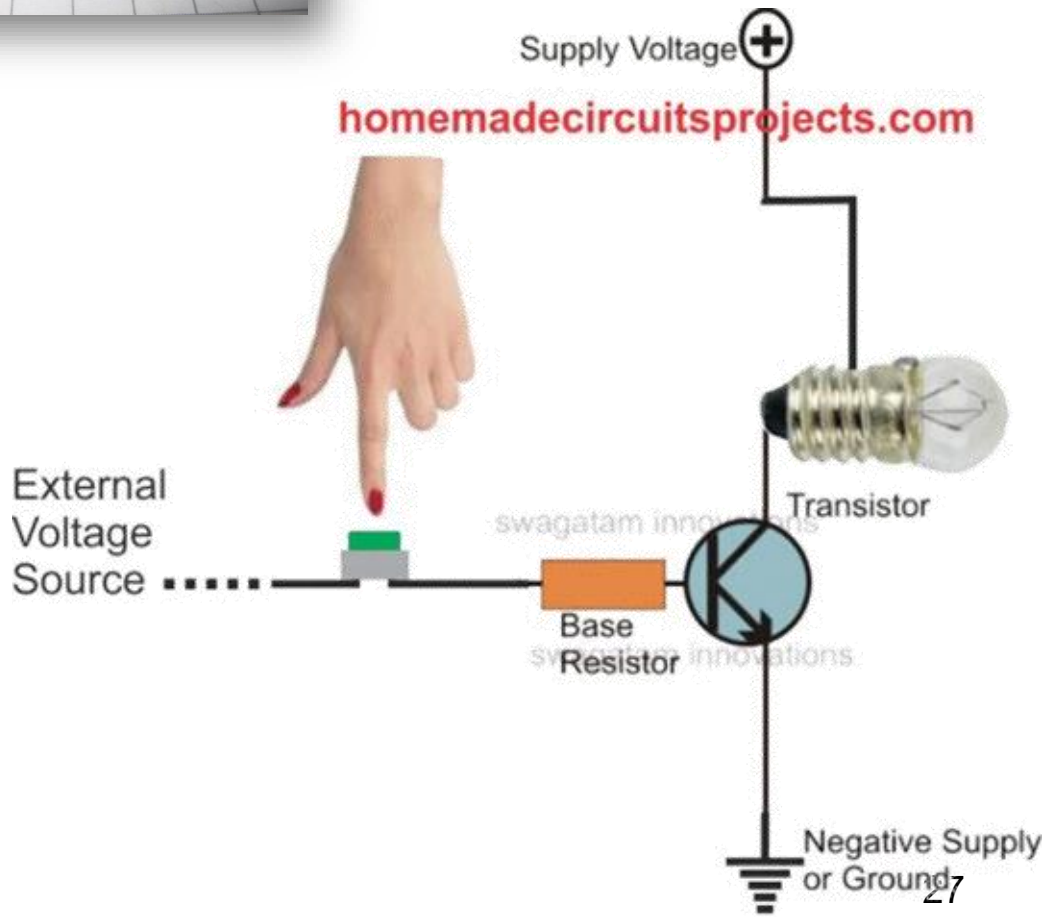
- Beschrieben durch 0/1
- “Strom an” / “Strom aus”







Source: Stefan Riepl von de.wikipedia.org/wiki/Relais



Darstellung

- Reihen von “0” und “1”
- Repräsentierung von allen „Daten“
  - Eingaben/Ausgaben
  - Code

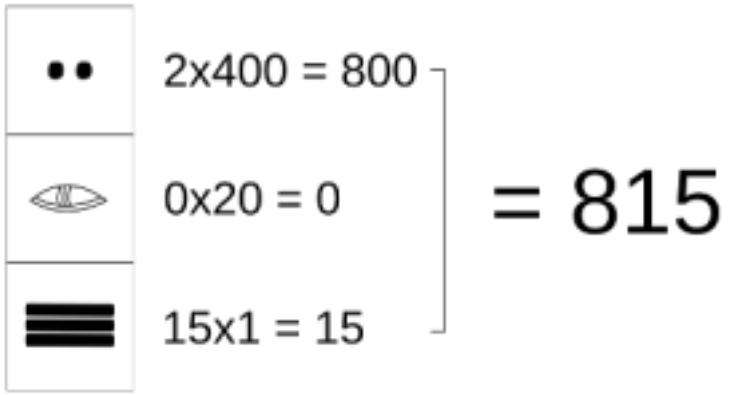
Zahlensysteme

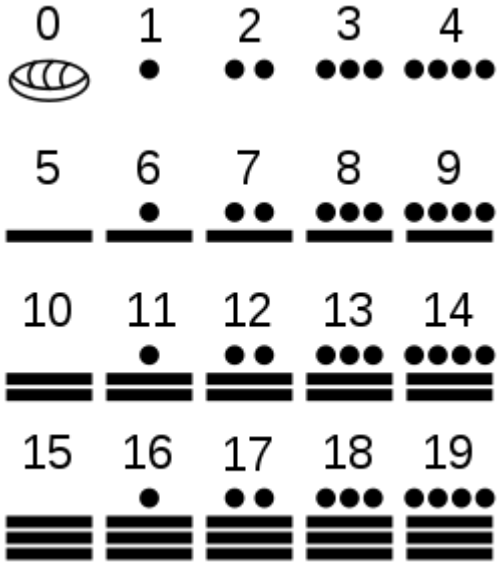
Wir sind das Dezimalsystem gewohnt

- $2019 = 2*10^3 + 0*10^2 + 1*10^1 + 9*10^0$
- 10 Zahlensymbole: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Basis: 10

Andere Kulturen haben andere Basen

- Beispiel: Maya – Basis: 20
- Konsequenz: 20 Zahlensymbole in 4 Gruppen zu je 5





From: https://www.mayaarchaeologist.co.uk/school-resources/maya-world/mayamaths/

Zahlensysteme

Wir sind das Dezimalsystem gewohnt

- $2019 = 2*10^3 + 0*10^2 + 1*10^1 + 9*10^0$
- 10 Zahlensymbole: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Basis: 10

Für Digital/Binärrechner ist das Binärsystem ideal

- $1101\ 1001_b = 1*2^7 + 1*2^6 + 0*2^5 + 1*2^4 + 1*2^3 + 0*2^2 + 0*2^1 + 1*2^0$
- $= 128 + 64 + 16 + 8 + 1 = 217$
- Zahlensymbole: 0, 1
- Basis: 2

Beispiel: $2019 = 111\ 1110\ 0011_b$

Hexadezimalsystem (Basis 16)

- $217 = D9_h = 13*16^1 + 9*16^0 = 1101\ 1001_b$
- Zahlensymbole: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- Vorteil: 1 Hex-Symbol = 4 Binär-Symbole

Binär- und Hexadezimal-Arithmetik

Arithmetik mit anderen Basen funktioniert wie mit Basis 10

```
  0 1 1 0 1 0 1 0
+ 0 0 1 1 1 0 0 1
-----------------
  1 0 1 0 0 0 1 1
```

```
    6 A
+   3 9
-------
    A 3
```

```
    1 0 6
+     5 7
---------
    1 6 3
```

Addition jeder einzelnen Stelle lässt sich durch eine einfache Tabelle beschreiben

Eingabe			Ausgabe	
Wert 1	**Wert 2**	**Übertrag**	**Wert**	**Übertrag**
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Gängige Wortlängen

Undefinierte Wortlängen sind schwer zu handhaben

- Datentransferlänge
- Speicherplatz
- Arithmetik

Rechensysteme arbeiten oft mit vorgegebenen Wortlängen

	Anzahl Bits	Werte	Alternativ-Name
Bit	1	0-1	
Nibble	4	0-15	
Byte	8	0-255	
Word	16	0-65.535	
Double Word	32	0-4.294.967.296	Word
Quad Word	64	$0\text{-}1{,}8 * 10^{19}$	Double Word

Negative Ganze Zahlen

Option: Nutzung eines Bits für +/-

Beispiel, höchstes Bit: -1 = 1000 0001

- Nachteil 1: Rechnung kompliziert
  -1 + 1 = 1000 0001 + 0000 0001 → 0 = 0000 0000
- Nachteil 2: 1000 0000 = 0000 0000

Meistgenutzte Lösung: Zweierkomplement

- Invertierung (Einerkomplement) + 1
  Beispiel: -1 = Inv(0000 0001) +1 = 1111 1110 +1 = 1111 1111
  Vorteil: -1 + 1 = 1111 1111 + 0000 0001 = 1 0000 0000 -> 0000 0000 = 0
  Allgemein: man kann die gleiche Addition anwenden!

Nicht vorgeschrieben -> gewählte Konvention
=> Entwurfsentscheidung beim Rechnerdesign

Gängige Wortlängen

	Bits	Unsigned	Signed
Bit	1	0-1	n/a
Nibble	4	0-15	-8 - 7
Byte	8	0-255	-128 - 127
Word	16	0-65.535	-32768 - 32767
Double Word	32	0-4.294.967.295	-2147483648 - 2147483647
Quad Word	64	$0\text{-}1{,}8 * 10^{19}$	$-9{,}2 * 10^{18} – 9{,}2 * 10^{18}$

Darstellungen von Realen Zahlen

Binäre Darstellungen sind prinzipiell auf ganze Zahlen beschränkt

- War schon für negative Zahlen ein Problem
- Benötigen eine spezielle Darstellung von realen Zahlen

Option: Fest-Komma Darstellung

- Tupel aus zwei Zahlen
  - Zahl vor dem Komma
  - Zahl nach dem Komma
- Repräsentierung durch Verknüpfung der zwei Zahlen



= 42.31

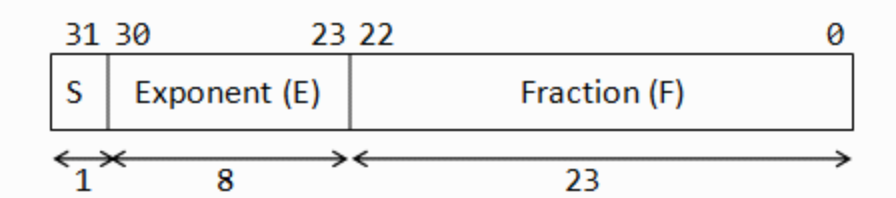
Kann numerisch problematisch sein

- Begrenzte Reichweite
- Unnötige Genauigkeit, die festgeschrieben ist

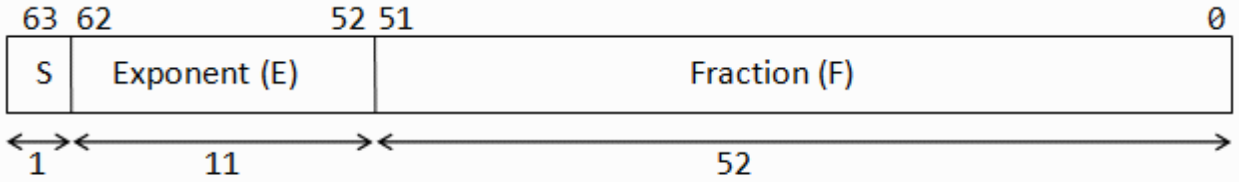
Gleitkommazahlen (Floating Point Numbers)

Für erhöhte Flexibilität: Aufteilung in Mantisse (auch Fraction) und Exponent (M*basis^E)

- Die meisten Systeme halten sich an IEEE-754
- Vorteil: breitere Reichweite
- Nachteil: ungleichmäßige Verteilung



32-bit Single-Precision Floating-point Number



64-bit Double-Precision Floating-point Number

Format: $(-1)^S * (1 + 0.F) * 2^{E-BIAS}$

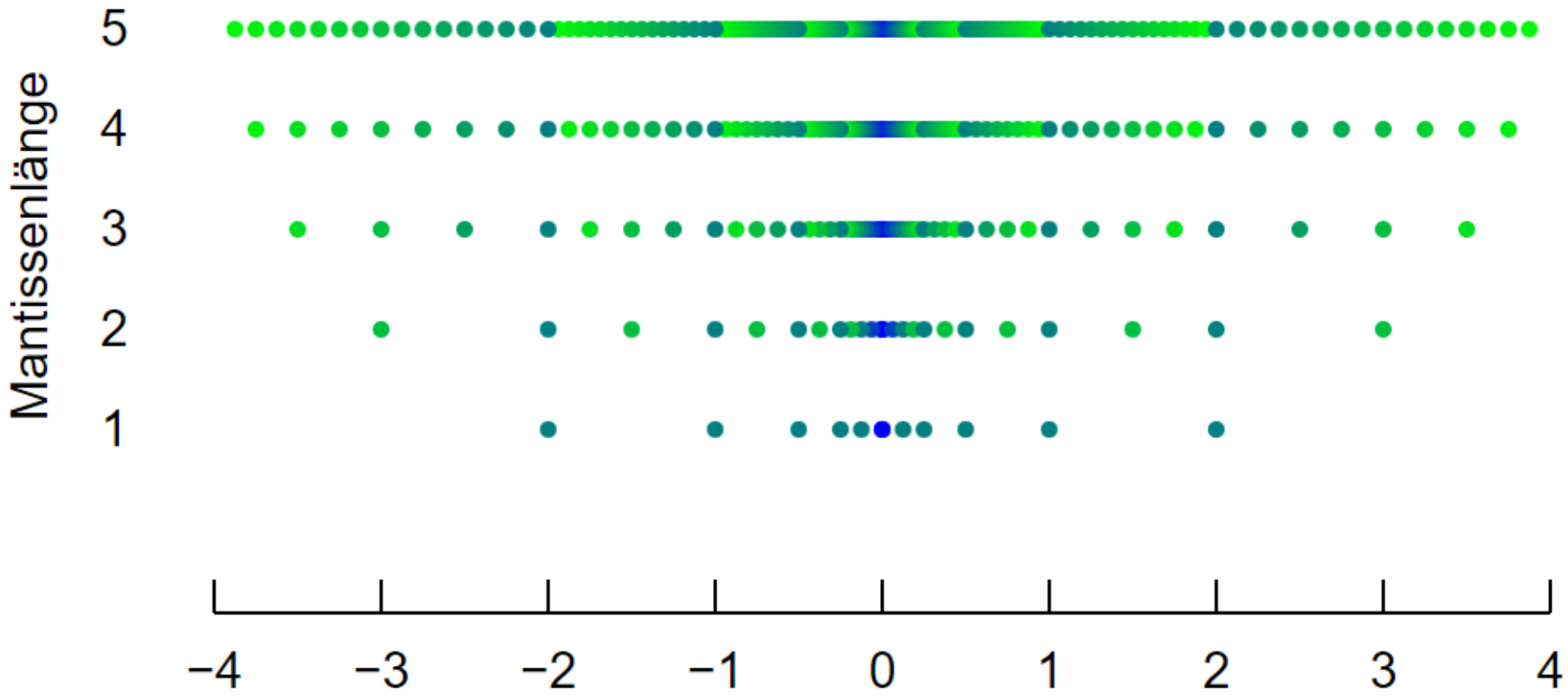
Ausführliches Tutorial (Ganze Zahlen und Gleitkommazahlen):
https://personal.ntu.edu.sg/ehchua/programming/java/datarepresentation.html
Mehr zu Gleitkommazahlen: https://agra.informatik.uni-bremen.de/doc/lehrmat/wise0304/ra/kap2.1b.pdf

Gleitkommazahlen (Floating Point Numbers)

	single precision	double precision
Vorzeichenstellen	1	1
Exponentenstellen	8	11
Mantissenstellen (ohne hidden Bit)	23	52
Bitstellen insgesamt	32	64
Bias	127	1023
Exponentenbereich	-126 bis 127	-1022 bis 1023
Darstellbare normalisierte Zahl mit kleinstem Absolutbetrag	2^{-126}	2^{-1022}
Darstellbare normalisierte Zahl mit größtem Absolutbetrag	$(1-2^{-24})\ 2^{128}$	$(1-2^{-53})\ 2^{1024}$
Darstellbare denormalisierte Zahl mit kleinstem Absolutbetrag	2^{-149}	2^{-1074}
Darstellbare denormalisierte Zahl mit größtem Absolutbetrag	$(1-2^{-23})\ 2^{-126}$	$(1-2^{-52})\ 2^{-1022}$

Von: https://agra.informatik.uni-bremen.de/doc/lehrmat/wise0304/ra/kap2.1b.pdf

Exakt darstellbare Gleitkommazahlen



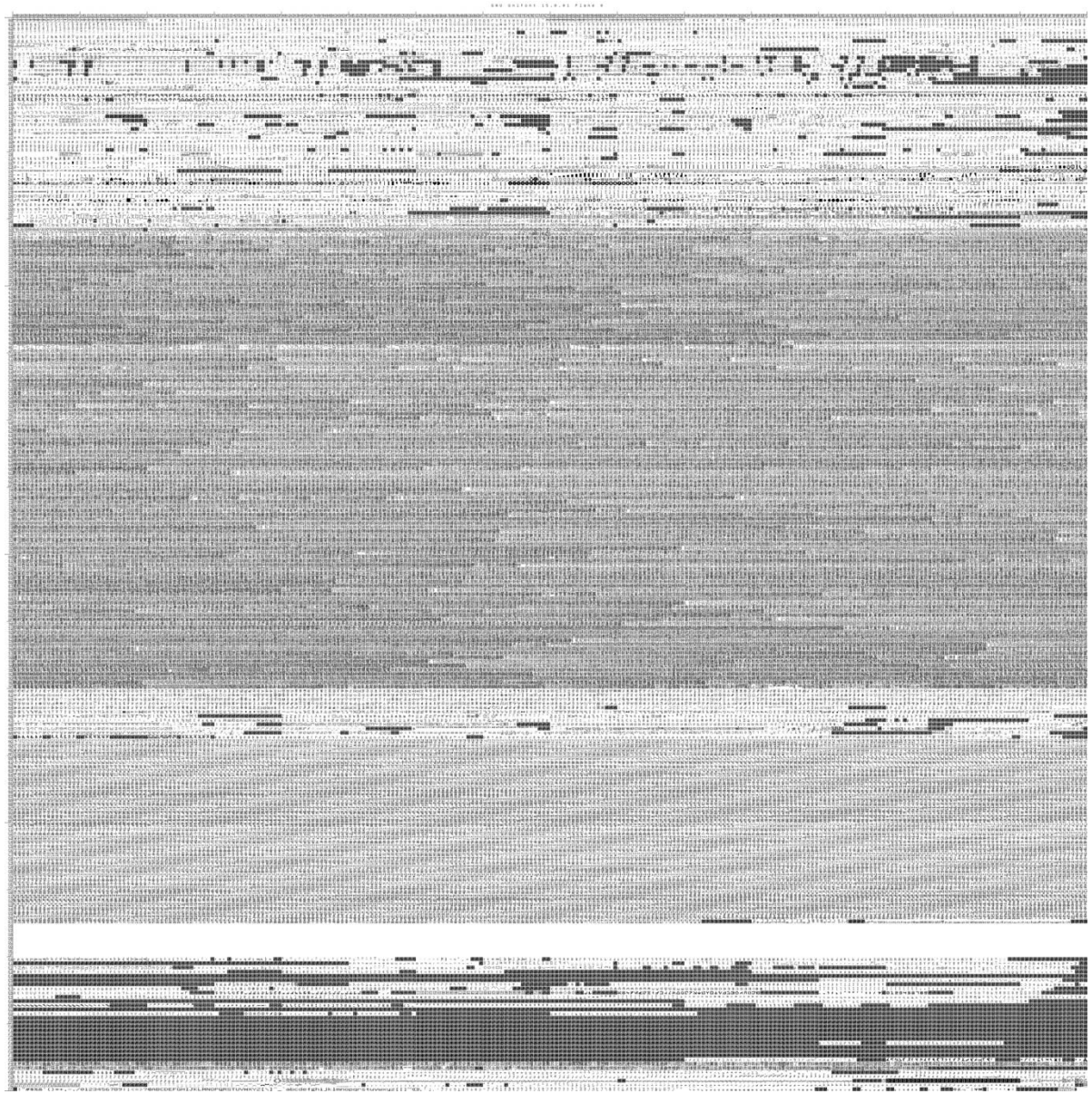
Warnung: alle Formate haben nur eine endliche Genauigkeit

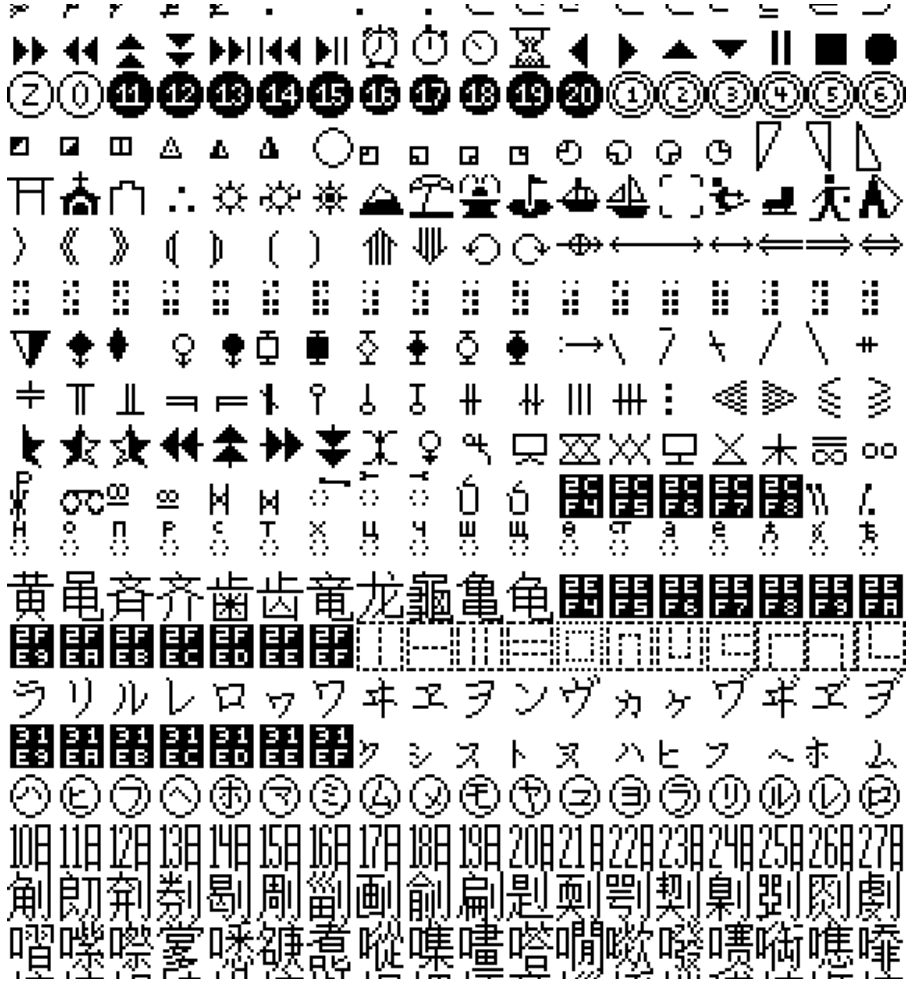
Von: https://de.wikipedia.org/wiki/Gleitkommazahl

Textzeichen müssen kodiert werden: ASCII Tabelle

Decimal	Hexadecimal	Binary	Octal	Char
0	0	0	0	[NULL]
1	1	1	1	[START OF HEADING]
2	2	10	2	[START OF TEXT]
3	3	11	3	[END OF TEXT]
4	4	100	4	[END OF TRANSMISSION]
5	5	101	5	[ENQUIRY]
6	6	110	6	[ACKNOWLEDGE]
7	7	111	7	[BELL]
8	8	1000	10	[BACKSPACE]
9	9	1001	11	[HORIZONTAL TAB]
10	A	1010	12	[LINE FEED]
11	B	1011	13	[VERTICAL TAB]
12	C	1100	14	[FORM FEED]
13	D	1101	15	[CARRIAGE RETURN]
14	E	1110	16	[SHIFT OUT]
15	F	1111	17	[SHIFT IN]
16	10	10000	20	[DATA LINK ESCAPE]
17	11	10001	21	[DEVICE CONTROL 1]
18	12	10010	22	[DEVICE CONTROL 2]
19	13	10011	23	[DEVICE CONTROL 3]
20	14	10100	24	[DEVICE CONTROL 4]
21	15	10101	25	[NEGATIVE ACKNOWLEDGE]
22	16	10110	26	[SYNCHRONOUS IDLE]
23	17	10111	27	[ENG OF TRANS. BLOCK]
24	18	11000	30	[CANCEL]
25	19	11001	31	[END OF MEDIUM]
26	1A	11010	32	[SUBSTITUTE]
27	1B	11011	33	[ESCAPE]
28	1C	11100	34	[FILE SEPARATOR]
29	1D	11101	35	[GROUP SEPARATOR]
30	1E	11110	36	[RECORD SEPARATOR]
31	1F	11111	37	[UNIT SEPARATOR]
32	20	100000	40	[SPACE]
33	21	100001	41	!
34	22	100010	42	"
35	23	100011	43	#
36	24	100100	44	$
37	25	100101	45	%
38	26	100110	46	&
39	27	100111	47	'
40	28	101000	50	(
41	29	101001	51	)
42	2A	101010	52	*
43	2B	101011	53	+
44	2C	101100	54	,
45	2D	101101	55	-
46	2E	101110	56	.
47	2F	101111	57	/
48	30	110000	60	0
49	31	110001	61	1
50	32	110010	62	2
51	33	110011	63	3
52	34	110100	64	4
53	35	110101	65	5
54	36	110110	66	6
55	37	110111	67	7
56	38	111000	70	8
57	39	111001	71	9
58	3A	111010	72	:
59	3B	111011	73	;
60	3C	111100	74	<
61	3D	111101	75	=
62	3E	111110	76	>
63	3F	111111	77	?
64	40	1000000	100	@
65	41	1000001	101	A
66	42	1000010	102	B
67	43	1000011	103	C
68	44	1000100	104	D
69	45	1000101	105	E
70	46	1000110	106	F
71	47	1000111	107	G
72	48	1001000	110	H
73	49	1001001	111	I
74	4A	1001010	112	J
75	4B	1001011	113	K
76	4C	1001100	114	L
77	4D	1001101	115	M
78	4E	1001110	116	N
79	4F	1001111	117	O
80	50	1010000	120	P
81	51	1010001	121	Q
82	52	1010010	122	R
83	53	1010011	123	S
84	54	1010100	124	T
85	55	1010101	125	U
86	56	1010110	126	V
87	57	1010111	127	W
88	58	1011000	130	X
89	59	1011001	131	Y
90	5A	1011010	132	Z
91	5B	1011011	133	[
92	5C	1011100	134	\
93	5D	1011101	135	]
94	5E	1011110	136	^
95	5F	1011111	137	_
96	60	1100000	140	`
97	61	1100001	141	a
98	62	1100010	142	b
99	63	1100011	143	c
100	64	1100100	144	d
101	65	1100101	145	e
102	66	1100110	146	f
103	67	1100111	147	g
104	68	1101000	150	h
105	69	1101001	151	i
106	6A	1101010	152	j
107	6B	1101011	153	k
108	6C	1101100	154	l
109	6D	1101101	155	m
110	6E	1101110	156	n
111	6F	1101111	157	o
112	70	1110000	160	p
113	71	1110001	161	q
114	72	1110010	162	r
115	73	1110011	163	s
116	74	1110100	164	t
117	75	1110101	165	u
118	76	1110110	166	v
119	77	1110111	167	w
120	78	1111000	170	x
121	79	1111001	171	y
122	7A	1111010	172	z
123	7B	1111011	173	{
124	7C	1111100	174	|
125	7D	1111101	175	}
126	7E	1111110	176	~
127	7F	1111111	177	[DEL]

Textzeichen müssen kodiert werden: Unicode UTF-16





Einführung in die Rechnerarchitektur (ERA)
IN0004
Abstraktionsebenen

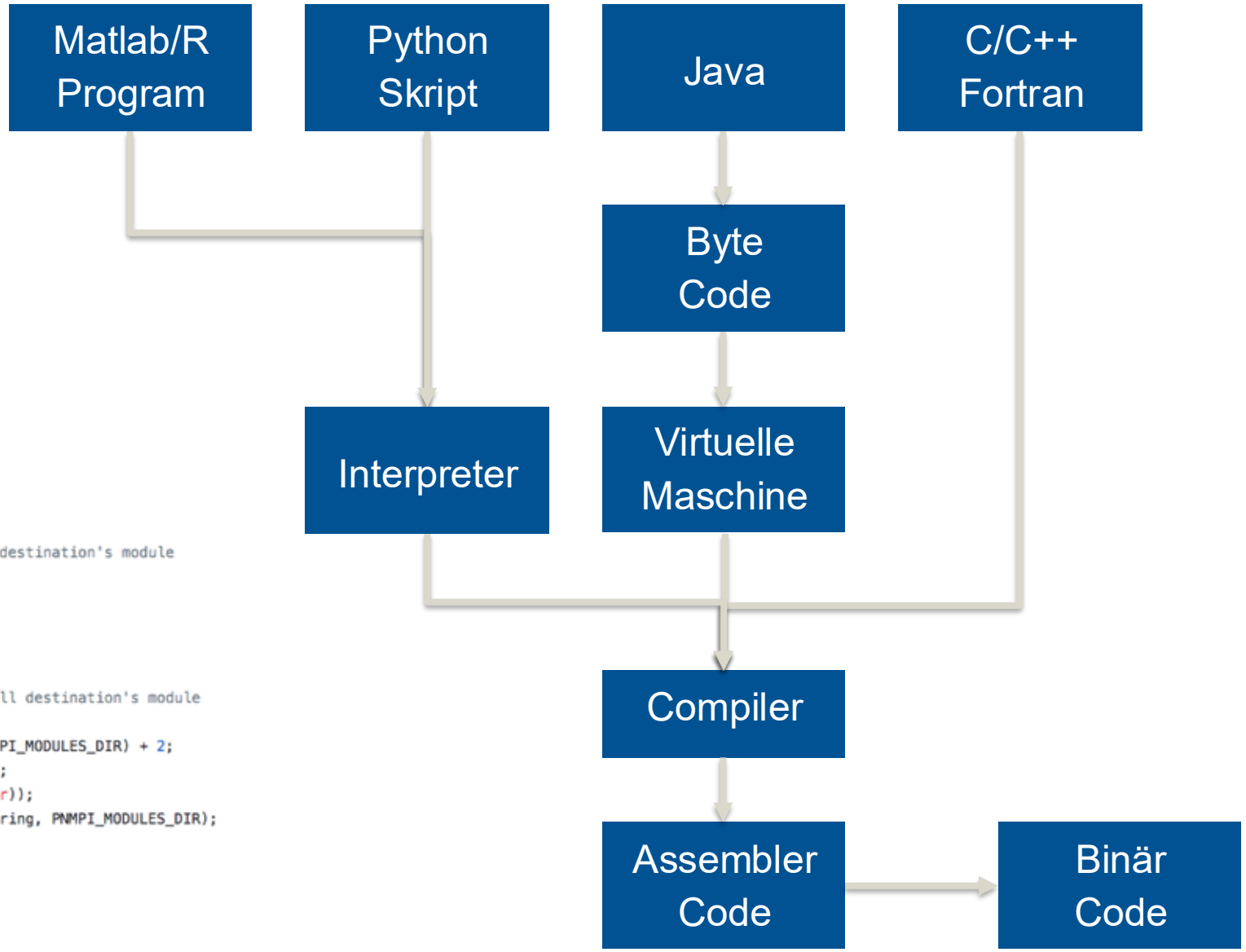
Martin Schulz
schulzm@in.tum.de
Chair for Computer Architecture and Parallel Systems
https://www.ce.cit.tum.de/caps/

Robert Wille
robert.wille@tum.de
Chair for Design Automation
https://www.cda.cit.tum.de/



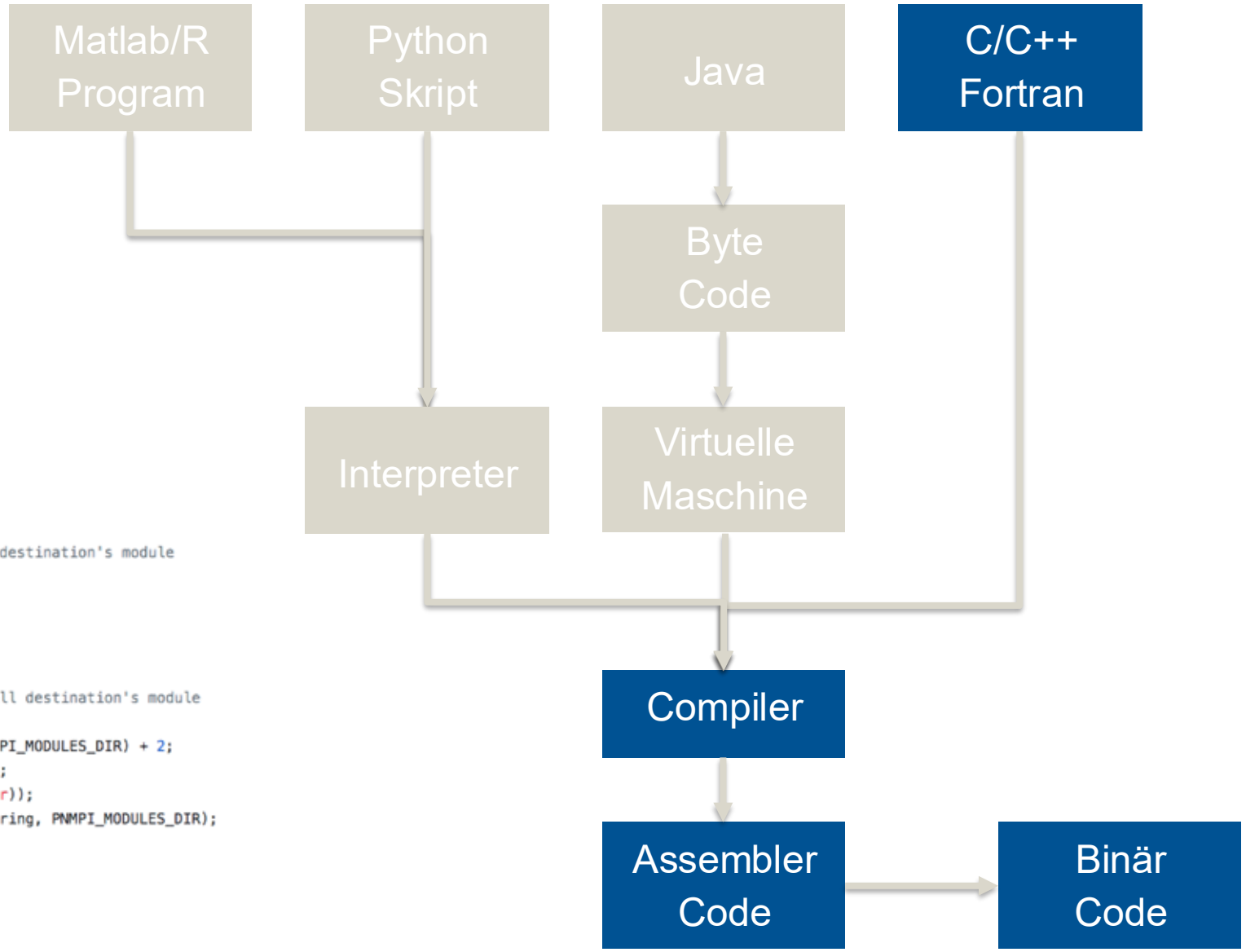
Programm-Abstraktion

```
217     int pos;
218     char *cmdargv[MAX_CMDARGS + 1];
219     int cmdargc, comment, i;
220
221     /* setup vars */
222
223     pnmpi_max_level = 0;
224     pnmpi_initialization_complete = 0;
225
226     /* set global defaults */
227     /* none at this moment */
228
229     modules.module = NULL;
230     modules.num = 0;
231     modules.numalloc = 0;
232     modules.pcontrol = PNMPI_PCONTROL_INT;
233
234     /* locate library */
235
236     lib_path_string = getenv("PNMPI_LIB_PATH");
237     if (lib_path_string == NULL)
238       {
239         // no user lib_path_string; just use the install destination's module
240         // path.
241         lib_path_string = PNMPI_MODULES_DIR;
242       }
243     else
244       {
245         // concat the user lib_path_string with the install destination's module
246         // path.
247         size_t len = strlen(lib_path_string) + strlen(PNMPI_MODULES_DIR) + 2;
248         const char *old_lib_path_string = lib_path_string;
249         lib_path_string = (char *)alloca(len * sizeof(char));
250         sprintf(lib_path_string, "%s:%s", old_lib_path_string, PNMPI_MODULES_DIR);
251       }
252     library_path = parse_path(lib_path_string);
253     DBGPRINT2("Library path is: %s", lib_path_string);
254
```



Programm-Abstraktion

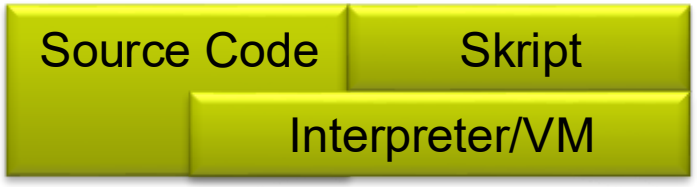
```
217     int pos;
218     char *cmdargv[MAX_CMDARGS + 1];
219     int cmdargc, comment, i;
220
221     /* setup vars */
222
223     pnmpi_max_level = 0;
224     pnmpi_initialization_complete = 0;
225
226     /* set global defaults */
227     /* none at this moment */
228
229     modules.module = NULL;
230     modules.num = 0;
231     modules.numalloc = 0;
232     modules.pcontrol = PNMPI_PCONTROL_INT;
233
234     /* locate library */
235
236     lib_path_string = getenv("PNMPI_LIB_PATH");
237     if (lib_path_string == NULL)
238       {
239         // no user lib_path_string; just use the install destination's module
240         // path.
241         lib_path_string = PNMPI_MODULES_DIR;
242       }
243     else
244       {
245         // concat the user lib_path_string with the install destination's module
246         // path.
247         size_t len = strlen(lib_path_string) + strlen(PNMPI_MODULES_DIR) + 2;
248         const char *old_lib_path_string = lib_path_string;
249         lib_path_string = (char *)alloca(len * sizeof(char));
250         sprintf(lib_path_string, "%s:%s", old_lib_path_string, PNMPI_MODULES_DIR);
251       }
252     library_path = parse_path(lib_path_string);
253     DBGPRINT2("Library path is: %s", lib_path_string);
254
```



Programm-Abstraktion

Eingabe: Code

- Kodiert als Text
- Nicht direkt ausführbar




```
a := 40;
b := 17;
do
    { a := a – b }
while(a >= 0) ;
a := -a;
```

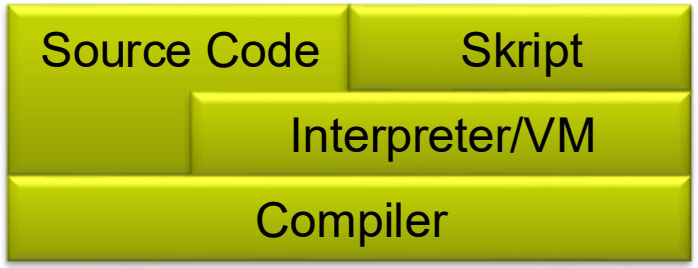
“Hochsprache”

Compiler: Transformation in Assembler-Sprache

Eingabe: Code
- Kodiert als Text
- Nicht direkt ausführbar

Assembler: Sprache auf niederer Abstraktionsebene
- Prozessor/System spezifisch
- Oft Familien von Systemen
- Einfachste Befehle, z.B. arithmetische Operationen

Eigenschaften
- Komplexität hängt vom Design ab
- Direkte Verwaltung von Ressourcen
  - Speicher
  - Argumente




```
loop_start:
        mov esi, [eax]
        imul esi, [ebx]
        add edx, esi
        add eax, 4
        add ebx, 4
        dec ecx
        jne loop_start
```

Assembler-Sprache

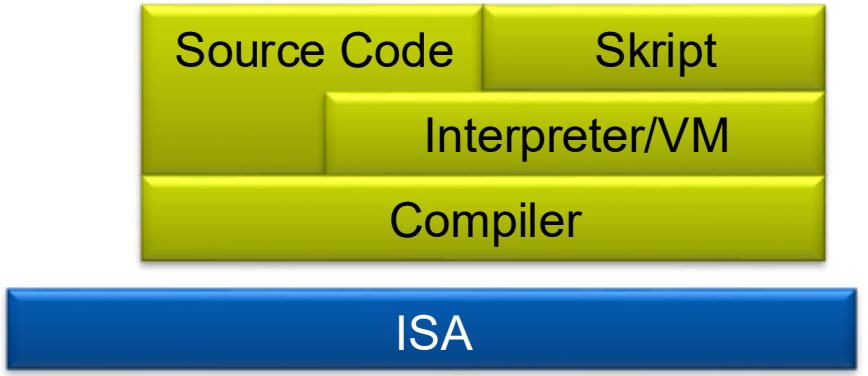
Instruction Set Architecture (ISA)

Datentypen
- Kodierung/Repräsentation

Speichermodell
- Was kann ich wo/wie speichern?

Sichtbarer Zustand des Prozessors
- Zugriff auf interne Daten
- Externe Schnittstellen

Verfügbare Befehle (Assembler)
- Ausführungssemantik



Einzige Programmiersprache, die ein Prozessor verstehen kann

=

Zentrale Nutzer-Schnittstelle

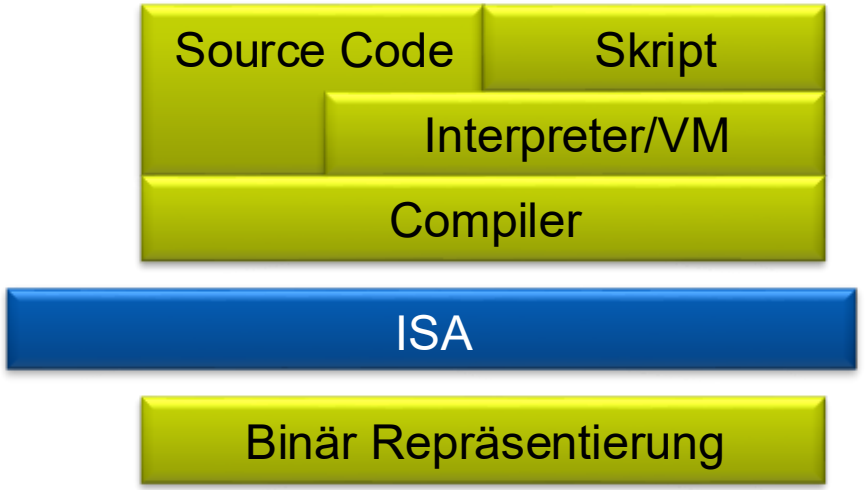
Übersetzung in Binärdarstellung

Datentypen
- Kodierung/Repräsentation

Speichermodell
- Was kann ich wo/wie speichern?

Sichtbarer Zustand des Prozessors
- Zugriff auf interne Daten
- Externe Schnittstellen

Verfügbare Befehle (Assembler)
- Ausführungssemantik
- Übersetzung in Binärform
- Art der Kodierung

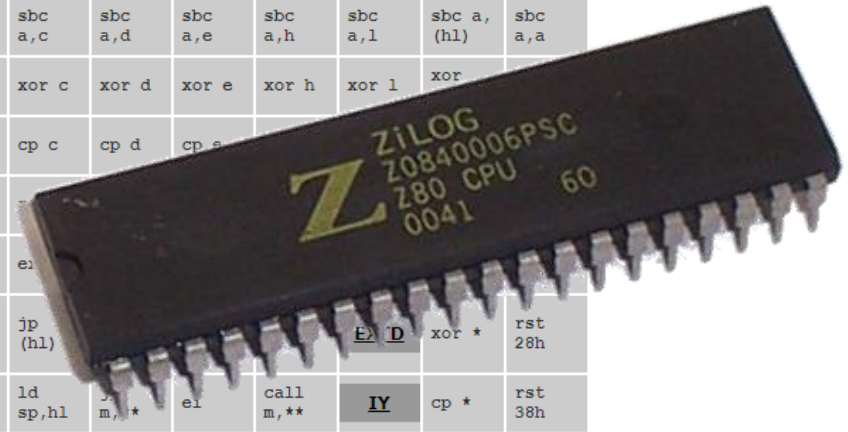




Beispiel Zilog Z80

Main instructions

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	nop	ld bc,**	ld (bc),a	inc bc	inc b	dec b	ld b,*	rlca	ex af,af'	add hl,bc	ld a, (bc)	dec bc	inc c	dec c	ld c,*	rrca
1	djnz *	ld de,**	ld (de),a	inc de	inc d	dec d	ld d,*	rla	jr *	add hl,de	ld a, (de)	dec de	inc e	dec e	ld e,*	rra
2	jr nz,*	ld hl,**	ld (**),hl	inc hl	inc h	dec h	ld h,*	daa	jr z,*	add hl,hl	ld hl, (**)	dec hl	inc l	dec l	ld l,*	cpl
3	jr nc,*	ld sp,**	ld (**),a	inc sp	inc (hl)	dec (hl)	ld (hl),*	scf	jr c,*	add hl,sp	ld a, (**)	dec sp	inc a	dec a	ld a,*	ccf
4	ld b,b	ld b,c	ld b,d	ld b,e	ld b,h	ld b,l	ld b, (hl)	ld b,a	ld c,b	ld c,c	ld c,d	ld c,e	ld c,h	ld c,l	ld c, (hl)	ld c,a
5	ld d,b	ld d,c	ld d,d	ld d,e	ld d,h	ld d,l	ld d, (hl)	ld d,a	ld e,b	ld e,c	ld e,d	ld e,e	ld e,h	ld e,l	ld e, (hl)	ld e,a
6	ld h,b	ld h,c	ld h,d	ld h,e	ld h,h	ld h,l	ld h, (hl)	ld h,a	ld l,b	ld l,c	ld l,d	ld l,e	ld l,h	ld l,l	ld l, (hl)	ld l,a
7	ld (hl),b	ld (hl),c	ld (hl),d	ld (hl),e	ld (hl),h	ld (hl),l	halt	ld (hl),a	ld a,b	ld a,c	ld a,d	ld a,e	ld a,h	ld a,l	ld a, (hl)	ld a,a
8	add a,b	add a,c	add a,d	add a,e	add a,h	add a,l	add a, (hl)	add a,a	adc a,b	adc a,c	adc a,d	adc a,e	adc a,h	adc a,l	adc a, (hl)	adc a,a
9	sub b	sub c	sub d	sub e	sub h	sub l	sub (hl)	sub a	sbc a,b	sbc a,c	sbc a,d	sbc a,e	sbc a,h	sbc a,l	sbc a, (hl)	sbc a,a
A	and b	and c	and d	and e	and h	and l	and (hl)	and a	xor b	xor c	xor d	xor e	xor h	xor l	xor	
B	or b	or c	or d	or e	or h	or l	or (hl)	or a	cp b	cp c	cp d	cp e				
C	ret nz	pop bc	jp nz,**	jp **	call nz,**	push bc	add a,*	rst 00h	ret z							
D	ret nc	pop de	jp nc,**	out (*),a	call nc,**	push de	sub *	rst 10h	ret c	ex						
E	ret po	pop hl	jp po,**	ex (sp),hl	call po,**	push hl	and *	rst 20h	ret pe	jp (hl)				E: D	xor *	rst 28h
F	ret p	pop af	jp p,**	di	call p,**	push af	or *	rst 30h	ret m	ld sp,hl	m,*	ei	call m,**	IY	cp *	rst 38h



Source: clrhome.org

Beispiel: Intel IA32 und x86_64

Komplexe Architektur

- Komplexe Kodierung
- Hunderte Befehle
- Hersteller Dokumentation





Einschub: Ausführung

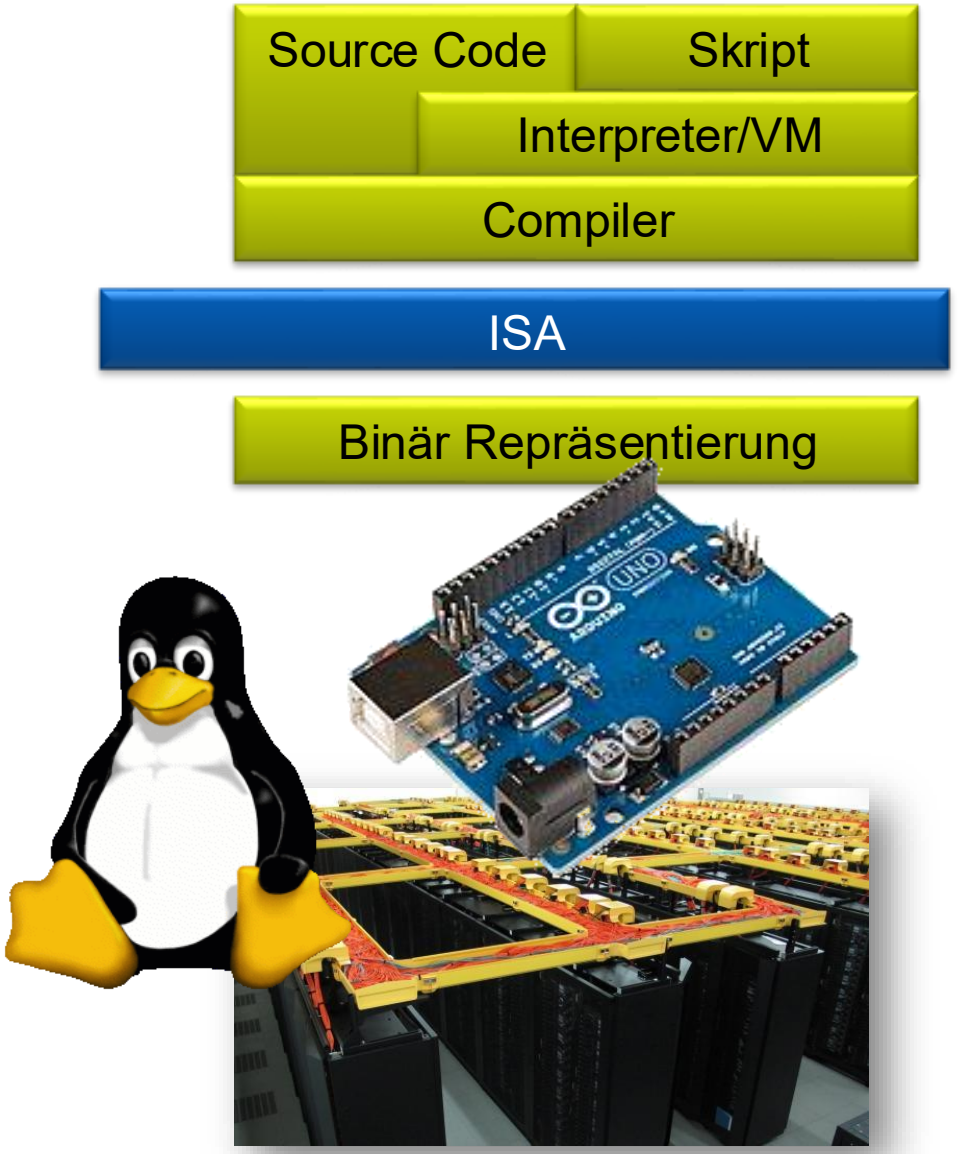
Binärprogramme können direkt
von einem Rechner verstanden werden

ABER: Wie starte ich sie?

(1) Direkte Ausführung
- Programm in Speicher laden
- Prozessor anstoßen

(2) Durch ein Betriebssystem
- OS an sich ein Programm
  - Lädt Programm in Speicher
  - Linker & Loader
- Transferiert (teilweise) Kontrolle

(3) Durch einen Ressourcenmanager

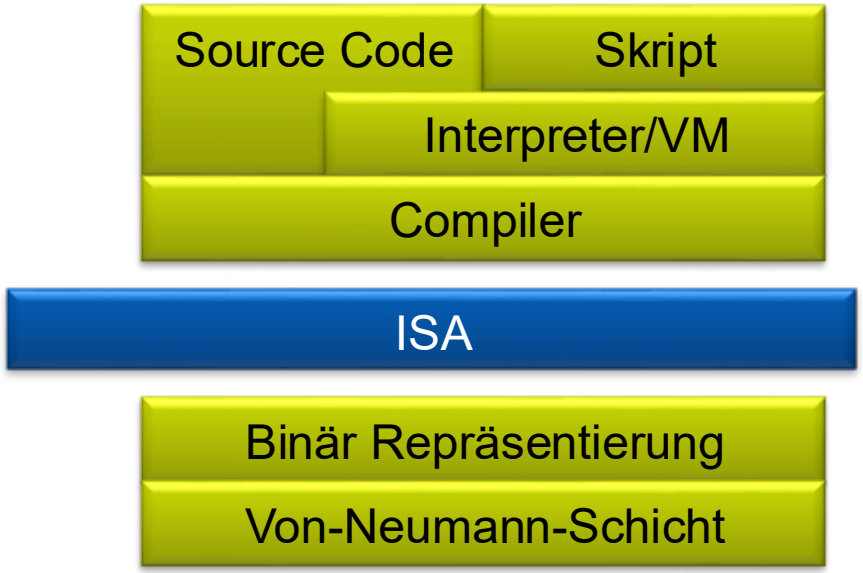


(John) Von Neumann Ansatz

Abstraktion eines Rechensystemes
- Wohldefinierte Komponenten
- Wohldefinierte Datenflüsse

Zentrale Eigenschaften
- Struktur des Rechners unabhängig vom bearbeiteten Problems
- Programm und Daten im gleichen Speicher
- Speicher ist in Zellen gleicher Größe unterteilt
- Programm ist eine Reihe von Befehlen

Bis heute gängigstes Konzept für das Design von Rechneranlagen





1903-1957

Mikro-Architekturen

ISAs auf Von-Neumann Ebene immer noch schwer direkt zu implementieren

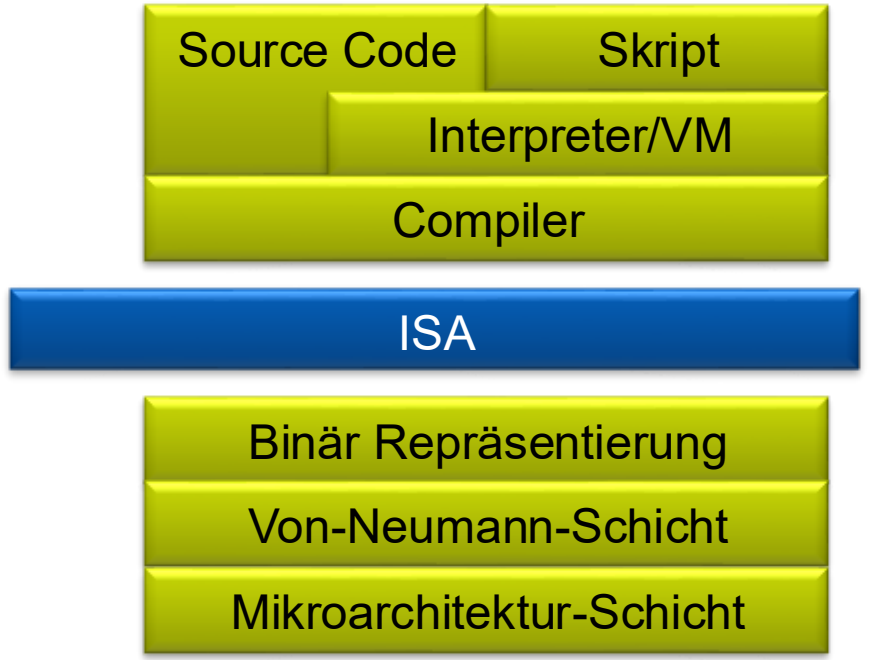
- Einzelne Befehle zu komplex
- Keine direkte Äquivalenz in Schaltungen/Hardware

Umsetzung/Transformation in einzelne “kleinere Befehle”

- Direkte Steuerung von Komponenten
- Einzelne Signale

Umsetzung hängt sehr vom System ab

- Oft intern und nicht veröffentlicht
- Eigene Programminterpretation
- Umwandlung in Micro-Ops
- Vereinfachte ISA
- …



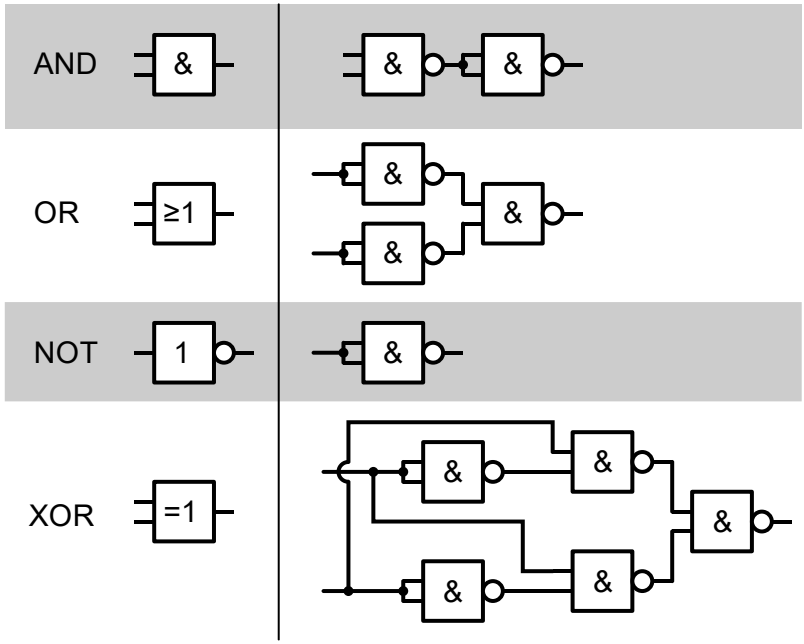
Gatter Schicht

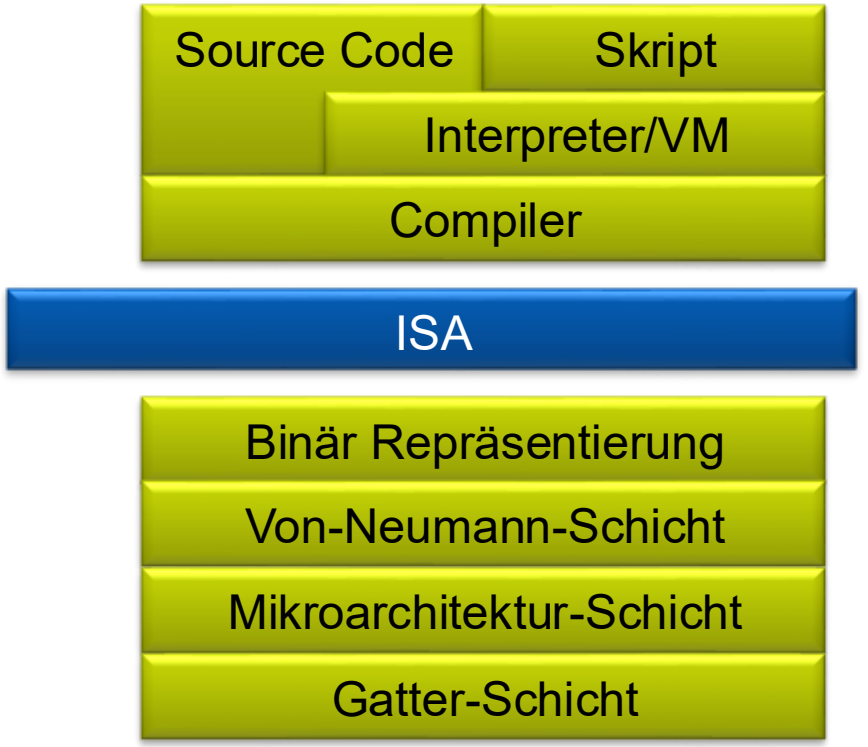
Einzelne Funktionseinheiten

- Elementare Boolesche Funktionen
- Direkte Umsetzung in “Schalter” möglich

Eigene Spezifizierungssprachen

- VHDL
- Verilog



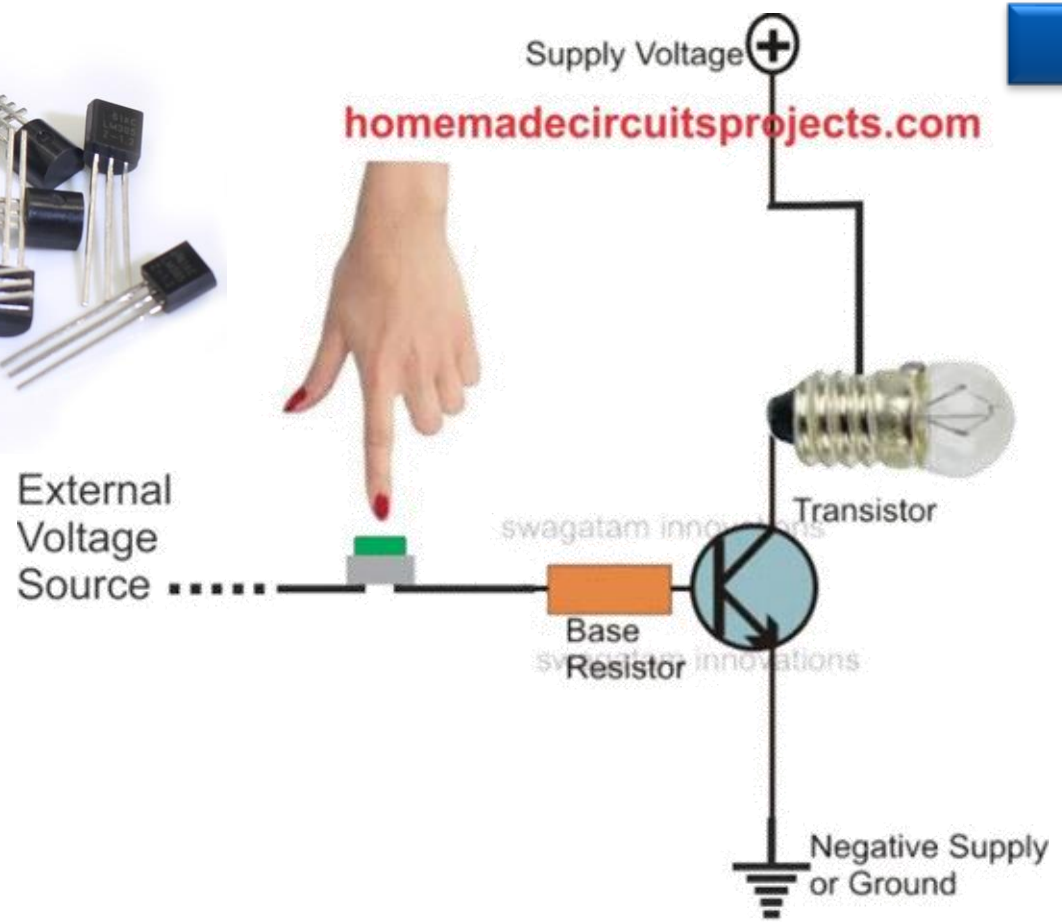


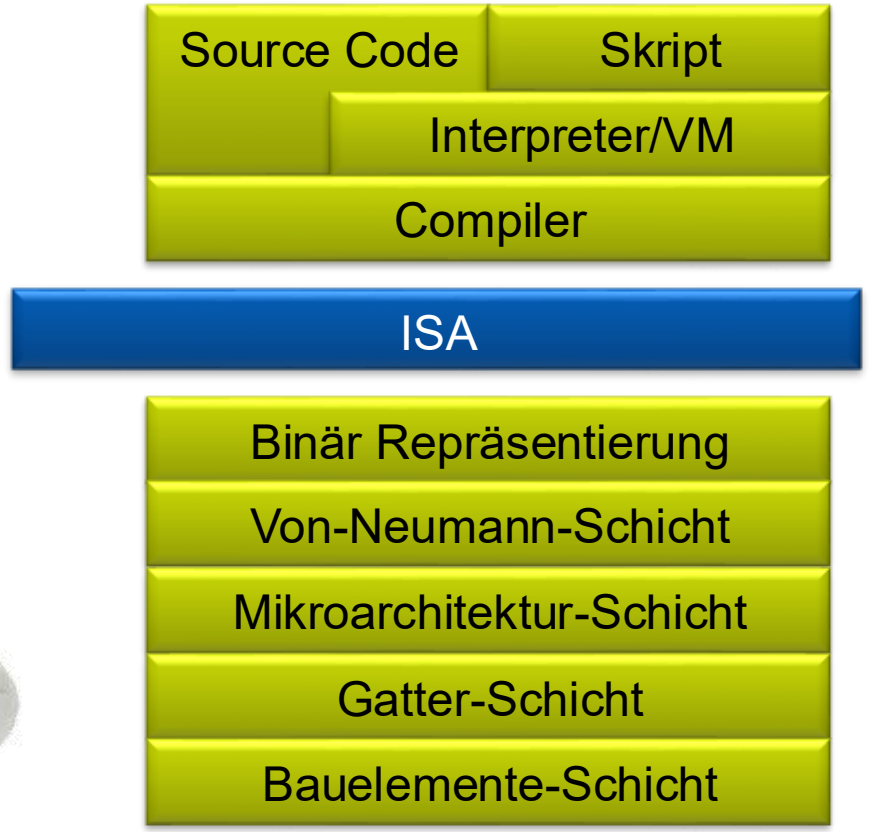
Bauelemente Schicht

Umsetzung einzelner Schaltungen in der Gatterschicht durch ”Schalter”

Implementierung in Transistoren



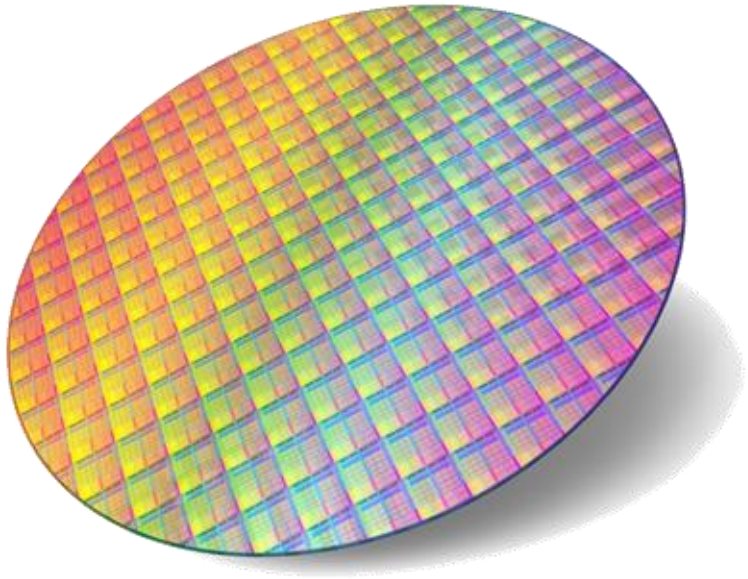


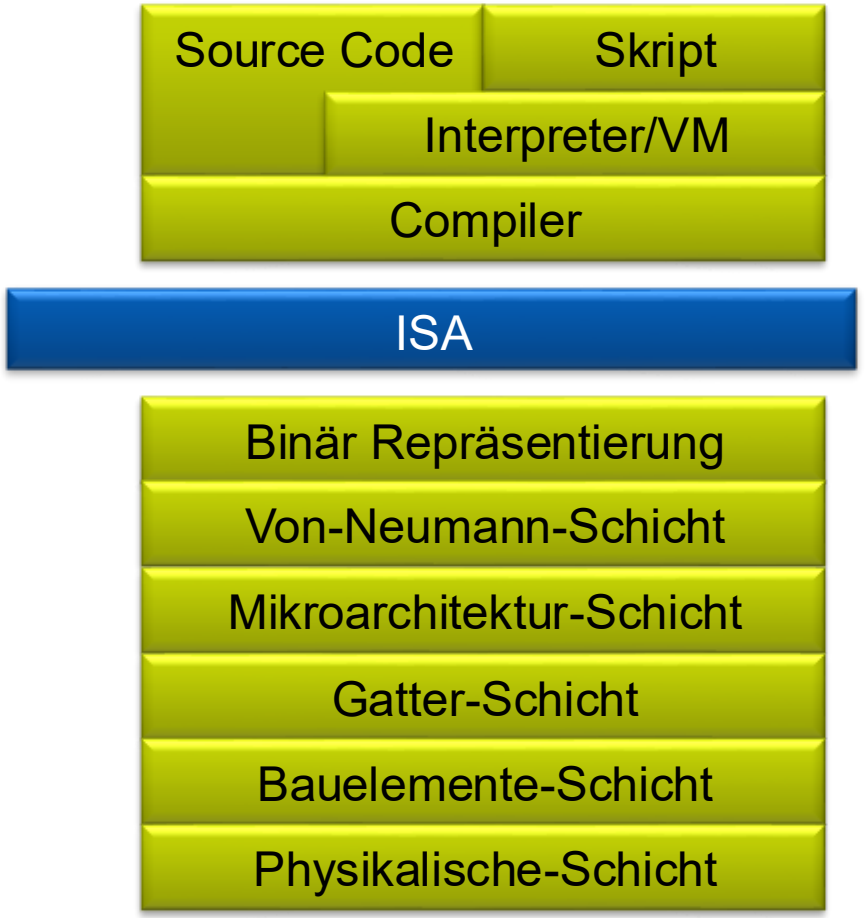


Physikalische Schicht

Implementierung einzelner Transistoren

Umsetzung ganzer Schaltungen durch geeignete Prozesstechnologien





Anmerkungen zu Abstraktionsebenen

Abstraktionen erlauben den Austausch von Ebenen

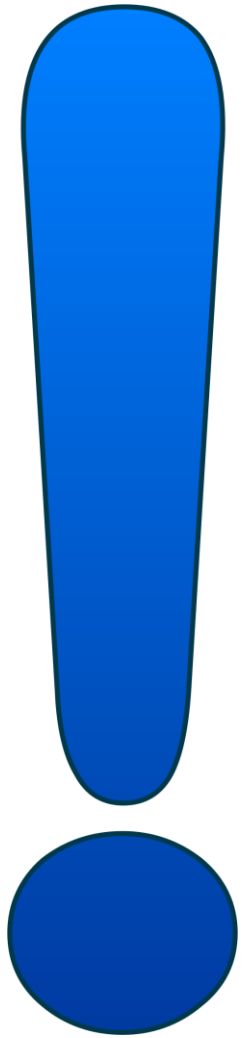
- Das gleiche C Programm kann auf verschiedenen ISAs laufen
- Neue ISAs haben eine Chance (Beispiel: RISC-V)
- Für x86-64 bestehen zwei Implementierungen (AMD und Intel)
- Neue physikalische Schichten beeinflussen obere Schichten nicht

Unterschiedliche Ziele können zu verschiedenen Design führen

- Numerische Simulationen brauchen hohe Gleitkomma Leistung
- Transaktionen (Datenbanken) brauchen Integer und I/O Leistung
- Prozessoren für Handys müssen Energieeffizient sein

Abstraktionen sind nicht “in Stein gemeißelt”

- Datenrepräsentationen können je Architektur verschieden sein
- Von-Neumann Konzept ist nur ein mögliches Konzept
- Wahl hat oft großen Einfluss auf die Effizienz der Architektur
- Basis für erweitertes Systemkonzept
  - E/A, Speicher, Peripherie, Netzwerke, …



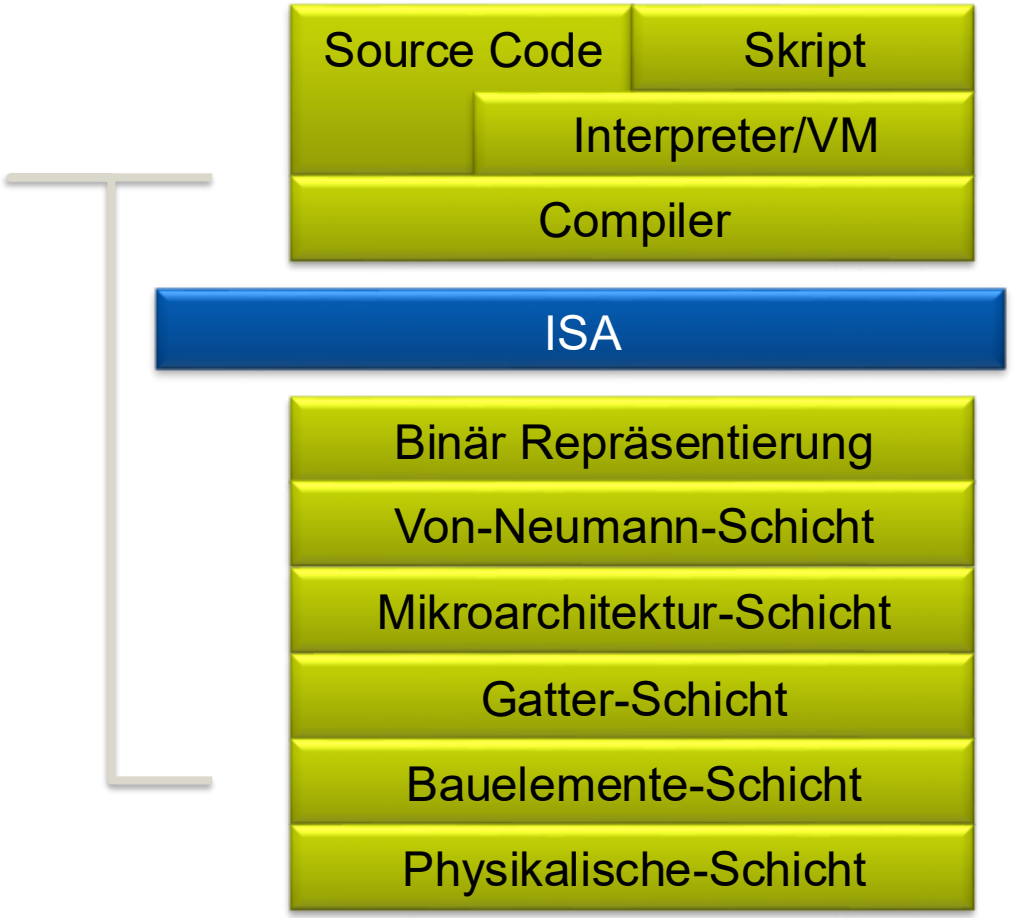
Ziele dieser Vorlesung

- Schichtung von Assembler …
  - Programmierung
  - Konventionen
  - ISA Differenzen und Design

- … über gängige Architekturen ...
  - Von-Neumann

- ... über Implementierungsoptionen ...
  - Mikroarchitekturkonzepte
  - Komponenten Design

- ... bis zur Auflösung in
  - Schaltungen
  - Einzelne Transistoren

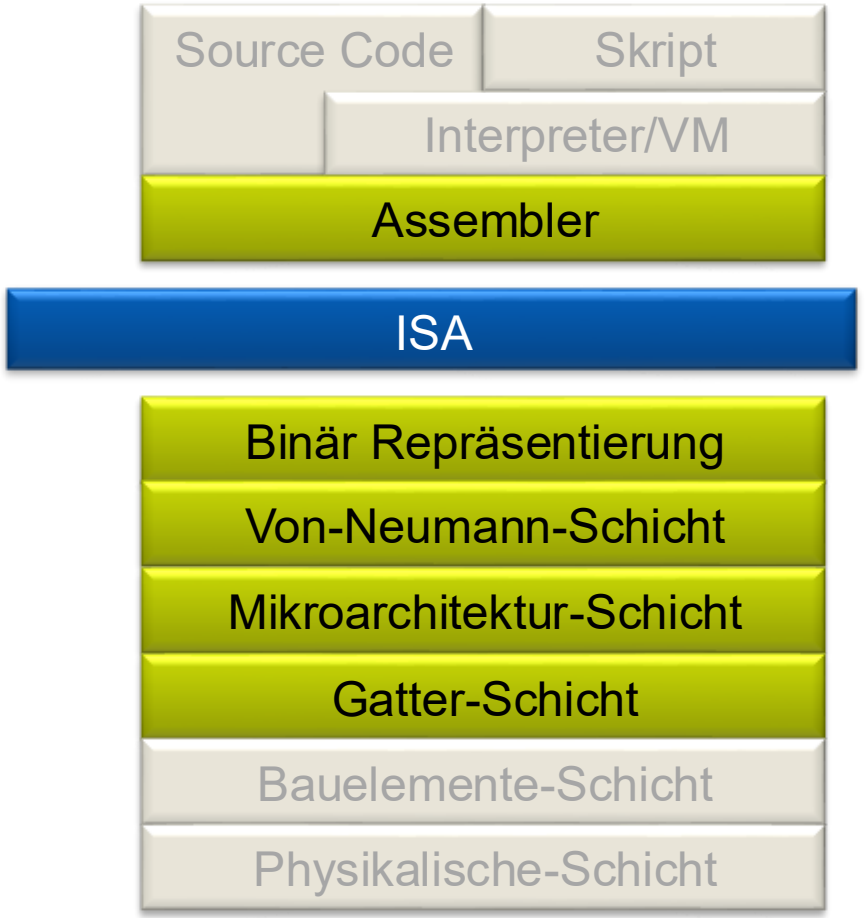


Einbettung dieser Vorlesung

Anbindung “nach oben”
- Programmiersprachen
- Parallel Programming
- Algorithmen und Datenstrukturen
- Betriebssysteme
- Compiler

Vertiefung
- Advanced Computer Architecture
- Performance Engineering
- Virtualization

Anbindung “nach unten”
- Circuit-Design (E-Technik)
- Prozesstechniken (ET/Physik)



Zusammenfassung

Inhalte Rechnerarchitektur
- Design von Prozessoren
- Systemarchitektur
  - V.a. Speichersysteme
- Umsetzung/Ausführung von Code

Stark geschichtete System
- Abstraktionsebenen
- ISA sichtbar für Programmierer
- Verständnis wichtig für Performanz

Warum?
- Weites Feld der RA/RA Design
- Verständnis von Ausführung
- Verständnis von Abwägungen
- Verständnis von Optimierungen

